PhishDrift

Evaluating Phishing-URL Detection Across Sources, Representations and Splits, and an Explainable Chrome-Extension Detector

Project-Based Learning final report

Submitted by **Your name** (Roll number)

Department, Institution

Guide: Guide name

Semester and year

Certificate

This is to certify that the Project-Based Learning report titled **“PhishDrift: Evaluating Phishing-URL Detection Across Sources, Representations and Splits, and an Explainable Chrome-Extension Detector”** is a record of work carried out by **Your name** (Roll number), Department, Institution, during Semester and year, under my guidance.

Guide: Guide name Signature: _____ Date: _____

Head of Department: _____ Signature: _____ Date: _____

Declaration

I, **Your name** (Roll number), declare that this report is my own work. The data come from public datasets and feeds named in the report, and its numbers come from files that scripts in the project repository write to the `reports/` folder, or from the project log, with the source named beside each. Where a result is estimated or not measured, the report says so. Other people's work is cited in the references.

Your name Signature: _____ Date: _____

Acknowledgement

Template, edit before submission. I thank my guide, Guide name, for direction and feedback during this project, the Department at Institution for the opportunity to carry out project-based learning, and the authors of the public datasets and tools used here, whose work made the study possible.

Contents

1. Introduction 6

2. Background and questions 7

3. Part I: design and method 8

4. Part I: results 10

5. Data and corpus 22

6. Feature engine and models 25

7. System: extension and backend 27

8. Evaluation of the detector 33

9. Limitations and what is not done 44

10. Conclusion and future work 46

References 47

Abstract

Phishing detectors are usually judged by one score on one benchmark. This project asks how far such scores can be trusted, and then builds a detector that is evaluated under the stricter rules the answer calls for. It has two parts. **Part I** is an evaluation study: seven questions about how a detector's score depends on URL form, source composition, the split and the legitimate pages it meets, tested across five phishing sources, five legitimate sources, four model families and ten domain-grouped splits, with the rule for calling a result consistent fixed before the run. **Part II** is the detector itself: a pooled, tiered phishing detector (URL model, page model, domain checks) shipped as a Chrome extension with a FastAPI backend, with allowlist and blocklist layers and plain-English reasons for every warning.

In Part I, the median score falls from 0.977 on raw URLs to 0.850 when a URL is reduced to its domain, and the fall appears in 100% of settings; transfer to datasets that share no source stays above chance. Source dependence holds in part (76% of settings) and the gap in false alarms on naturally browsed legitimate pages holds in part (76%): at the 1% threshold 15.2% of those pages are flagged against 1.0% on the model's own test side. The share of the fall that sits in the host name does not hold across cells and depends on how the benchmark is prepared, and the larger split-to-split spread of the normalised score does not hold. Masking hosting platforms showed no measurable effect in an exploratory test (Section 4.14), and a second condition without the data cleaning changes the answers to some questions (Section 4.13). Each result is stated as behaviour under one frozen design, not as a general claim.

1. Introduction

1.1 Motivation

Phishing sites steal passwords and payment details, and new ones appear every hour. Machine-learning detectors are an obvious defence, and many published ones report accuracy above 99% on a public dataset. A detector that reaches that on one dataset can still fail on the pages a person actually browses. In this project the median AUC falls from 0.977 on raw URLs to 0.850 once URLs are reduced to their domain, and at the 1% threshold set on a benchmark, 15.2% of ordinary legitimate pages are flagged against 1.0% on the benchmark itself.

1.2 Objectives

1. **Measure** how far a detector's score depends on URL form, source composition, the split and the legitimate pages it meets, across many datasets and several model families (Part I).
2. **Build** a practical detector, evaluated only under protocols that guard against those failures, and ship it as a working Chrome extension with a backend (Part II).
3. **Report** every number from a script that writes to `reports/`, with fixed seeds, and say plainly where a result is measured, inferred or not tested.

1.3 What is new

A plain "classifier on a static phishing dataset" is common. Novelty here is in the protocol: domain-grouped splits repeated ten times, leave-one-source-out and out-of-distribution false-alarm reporting, negative controls with shuffled labels, URL-depth checks, and tiered models whose missing inputs never reveal the data source; then a deployed, explainable system built on that protocol.

1.4 Scope and structure

Chapter 2 gives the background and the questions. Chapters 3 and 4 are Part I, the evaluation study. Chapters 5 to 9 are Part II: data, features and models, the extension and backend, the evaluation of the detector, and the limitations. Chapter 10 concludes. Results from Part I are not used as evidence for Part II, or the reverse.

2. Background and questions

2.1 Why one benchmark score misleads

A phishing-URL detector is normally scored once, on one dataset, with one split. Three things can make that number unrepresentative: the *form* of the URL (a raw URL carries path, query and host quirks that differ between sources), the *composition* of the data (which sources supply each class, and how deep their URLs are), and the *split* (which domains happen to land in the test side). A fourth gap opens at deployment: the legitimate pages a person browses are not the legitimate pages of the benchmark. This study measures each of these directly.

2.2 Questions asked

The study answers seven questions, one for each way a single score can mislead:

- **Q1** Does the AUC fall from raw URLs (R1) to the normalised `https://www.<registrable domain>` form (R4)?
- **Q2** Does most of that fall sit in host and subdomain structure rather than in path and query?
- **Q3** Does swapping one source (phishing or legitimate) move R4 AUC?
- **Q4** Is R4 AUC much less stable across splits than raw AUC?
- **Q5** Does masking hosting-platform identity raise R4 AUC?
- **Q6** Does a model still beat chance on cells that share no source with its training data?
- **Q7** Does a model flag far more naturally browsed legitimate pages than its own test side?

A second, exploratory part repeats the representation idea on page HTML (Section 4.11). It gives no verdicts.

2.3 Related datasets and tools

The study and the detector draw on public datasets (PhreshPhish, PhiUSIIL, URL-Phish, PhishStorm, Phishpedia, Phish-Blitz) and live feeds (OpenPhish, Phishing.Database). Only the URL, label, date and page HTML are read from any of them; no dataset's precomputed feature columns are used, because one such column (a similarity index) alone identifies PhiUSIIL's legitimate class. Features follow the published definitions of Hannousse and Yahiouche and of PhiUSIIL, re-implemented in one engine. Models are LightGBM, XGBoost, logistic regression and a character n-gram TF-IDF linear model.

3. Part I: design and method

3.1 Sources

URL, label and (for Part B) saved HTML are the only fields read from any source; no dataset's precomputed feature columns are used. Every population is capped at 10,000 URLs and at most 5 URLs per registrable domain, so URL counts and domain counts are reported separately (Table 4.1). Registrable domains are disjoint across sources, domains shared by several phishing sources are removed, and the build fails if any URL-depth bucket is more than 90% one class.

Table 4.1. Populations after the caps (`phishguard_run/populations.parquet`).

Part	Population	URLs	domains
A	benign:crawl	4964	1204
A	benign:phishstorm	10000	7753
A	benign:phiusiil	10000	9996
A	benign:phreshphish	10000	9475
A	benign:urlphish	10000	9773
A	phish:phishpedia	8578	5897
A	phish:phishstorm	10000	7154
A	phish:phiusiil	10000	9149
A	phish:phreshphish	10000	8174
A	phish:urlphish	4307	3590
B	benign:crawl	4826	1177
B	benign:phreshphish	10000	9491
B	phish:phishpedia	3937	2980
B	phish:phreshphish	10000	7371

3.2 Representations

R1 is the raw URL. R2 keeps the host only (path, query and fragment removed). R3 additionally strips `www`; here R3 is identical to R2 because the feature engine drops a leading `www` from every URL, so the `www` step is zero by construction. R4 is `https://www.<registrable domain>`.

3.3 Models and splits

LightGBM, XGBoost, logistic regression on the 99 URL features, and character n-gram TF-IDF with logistic regression. Ten splits per cell are grouped by registrable domain (train 70%, calibration 10%, test 20%; a deterministic hash of seed, split and domain). Fitted features (TLD risk, bigram likelihood) are refitted on each split's training side. Uncertainty comes from a domain-cluster bootstrap with 200 draws per split. Transfer is tested only on cells sharing no source with the training cell.

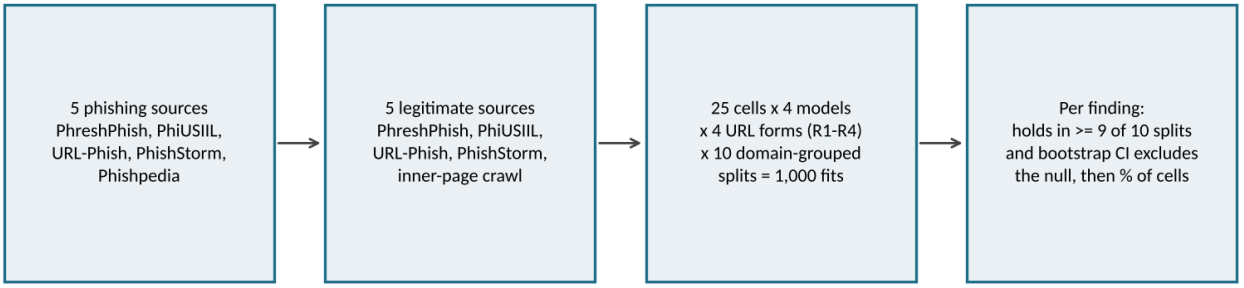


Figure 4.1. Study design. Seven questions are tested across 25 source cells, four models, four URL forms and ten splits.

3.4 The rule for calling a result consistent

A setting (one cell and model, or one pair of cells) *holds* when the direction is the same in at least 9 of 10 splits and the pooled bootstrap 95% interval excludes the null. A question's answer **holds** if it holds in at least 80% of its testable settings, **holds in part** at 50 to 80%, and **does not hold** below 50%; with fewer than 3 testable settings it is **not testable**. A setting is testable with at least 500 URLs and 100 domains per class. The rule and `configs/phishguard.yaml` were frozen at commit `082b058` before any result was seen. "Does not hold" describes this design's data; it is not a claim that the effect is absent elsewhere.

4. Part I: results

4.1 Q1: representation sensitivity (holds)

AUC falls from R1 to R4 in all 100 model-cell settings. The median R1 AUC is 0.977, R2/R3 0.877 and R4 0.850; the smallest drop in any setting is 0.024. All four model families show it (Figure 4.3).

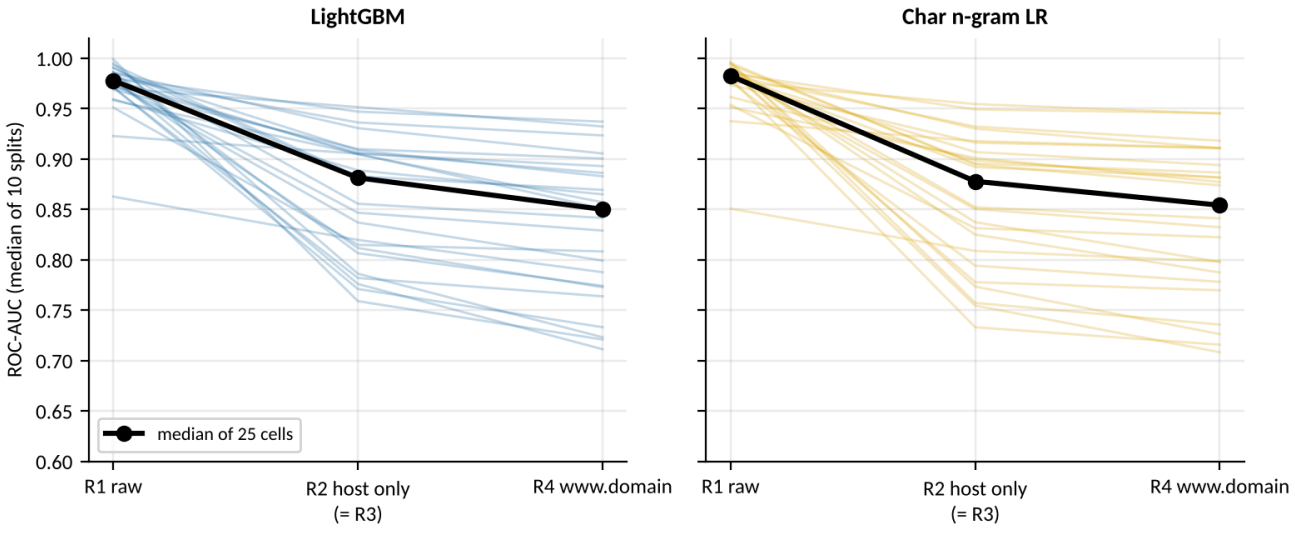


Figure 4.2. Representation ladder, 25 cells. Thin lines are cells; the thick line is the median.

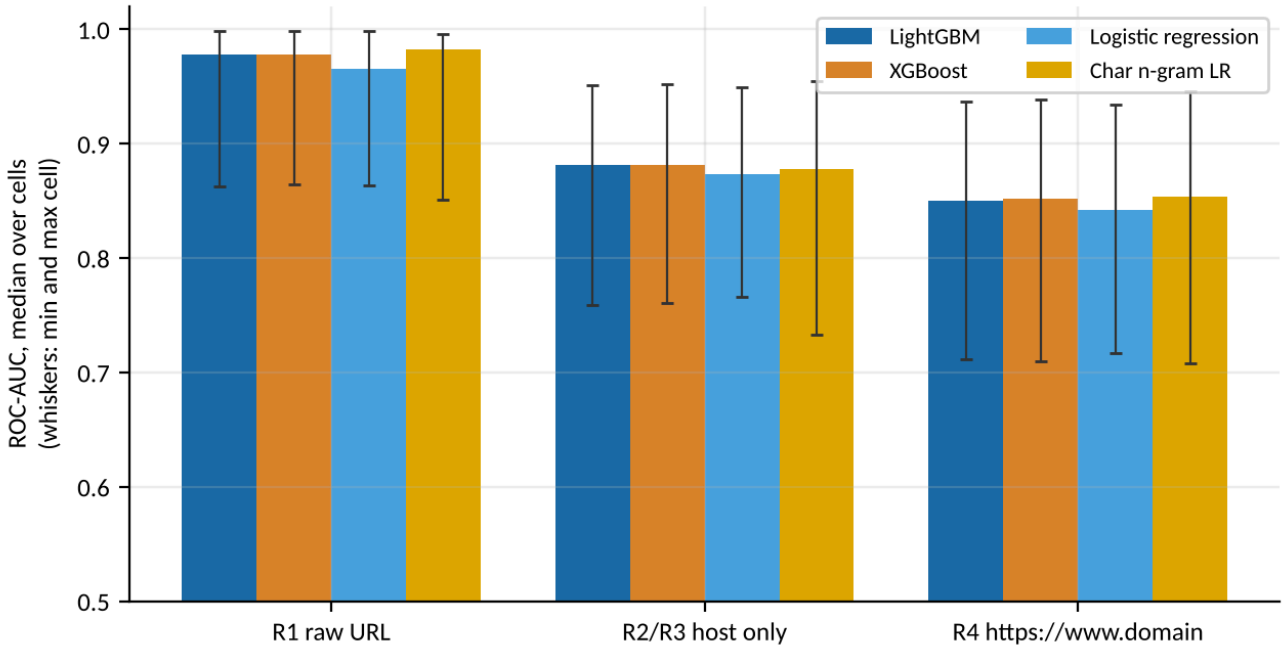


Figure 4.3. Median AUC by representation and model, with the range across the 25 cells.

Table 4.2. Mean AUC over the four models, by cell.

phishing vs legitimate	R1	R2 (= R3)	R4
phishpedia vs crawl	0.972	0.764	0.730
phishpedia vs phishstorm	0.975	0.846	0.831
phishpedia vs phiusiil	0.998	0.761	0.727
phishpedia vs phreshphish	0.968	0.786	0.769
phishpedia vs urlphish	0.993	0.885	0.872
phishstorm vs crawl	0.980	0.773	0.712
phishstorm vs phishstorm	0.972	0.837	0.798
phishstorm vs phiusiil	0.995	0.784	0.725
phishstorm vs phreshphish	0.978	0.815	0.776
phishstorm vs urlphish	0.991	0.882	0.854
phiusiil vs crawl	0.983	0.802	0.775
phiusiil vs phishstorm	0.988	0.853	0.841
phiusiil vs phiusiil	0.860	0.817	0.792
phiusiil vs phreshphish	0.948	0.818	0.812
phiusiil vs urlphish	0.923	0.906	0.895
phreshphish vs crawl	0.976	0.900	0.858
phreshphish vs phishstorm	0.983	0.929	0.907
phreshphish vs phiusiil	0.958	0.906	0.860
phreshphish vs phreshphish	0.955	0.892	0.870
phreshphish vs urlphish	0.970	0.952	0.935
urlphish vs crawl	0.974	0.904	0.887
urlphish vs phishstorm	0.977	0.934	0.923
urlphish vs phiusiil	0.985	0.909	0.886
urlphish vs phreshphish	0.967	0.911	0.903
urlphish vs urlphish	0.980	0.946	0.939

4.2 Q2: where the drop sits (does not hold)

The host and subdomain step (R2 to R4) carries a median share of 0.21; the larger part comes from removing path and query (R1 to R2). The statement “most of the fall sits in host structure” holds in 0 of 100 settings. The feature set, depth balancing and the per-domain cap can all change where the information sits; the main study does not choose among those explanations.

Exploratory follow-up (not part of the frozen verdict). Re-running PhiUSIIL alone with one cleaning step added at a time (Section 4.10) shows the host share of the drop depends on how the data was prepared. On raw PhiUSIIL through our loader the host step carries 0.76 of the drop for XGBoost and 0.42 for the character model; on the study cell, after the shared-domain removal, depth balancing and caps, it carries 0.36 and 0.21. A host-dominated drop is therefore visible on

the raw benchmark for XGBoost and fades as the benchmark is cleaned. That is consistent with the pattern being tied to how the raw benchmark was built, but it is one dataset and two models, so it is stated as a lead, not a conclusion.

Table 2b. Host and subdomain share of the R1 to R4 drop, PhiUSIIL alone, by cleaning stage (5 splits).

PhiUSIIL cleaning stage	Char n-gram host share	XGBoost host share
S0 raw PhiUSIIL	0.416	0.762
S0b raw, sampled to 10,000 per class	0.300	0.722
S1 + domains shared with other sources removed	0.305	0.514
S2 + pool URL rules	0.271	0.446
S3 + study caps (5 per domain, 10,000 per class) = study cell	0.212	0.359

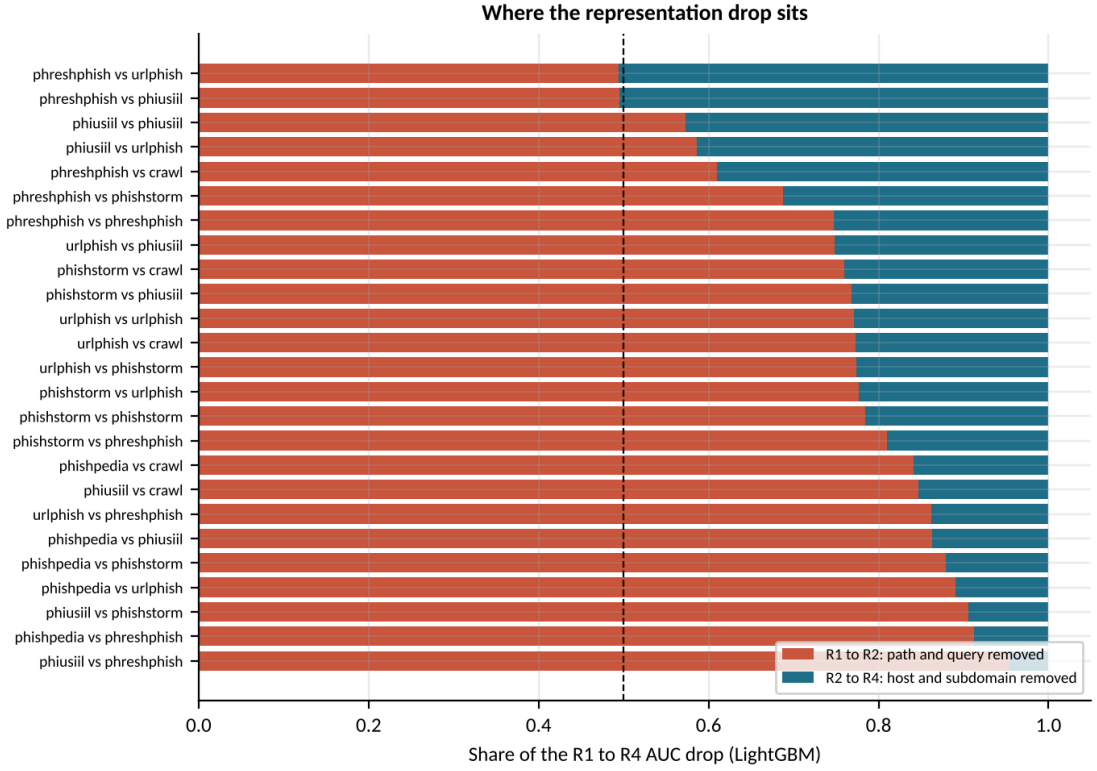


Figure 4.4. Share of the R1 to R4 drop by step, LightGBM, per cell.

4.3 Q3: source dependence (holds in part)

Swapping one source while holding the other fixed moves R4 AUC by a median 0.058 (phishing source swapped 0.050, legitimate source swapped 0.067). The interval excludes zero in 76% of 400 swaps. After reweighting the test side to equalise URL depth across classes it falls to 53%, so part of the source effect is URL depth. The heatmap shows the pattern: R4 AUC ranges from 0.71 to 0.94; against URL-Phish legitimate URLs every phishing source scores highest (0.85 to 0.94), and the lowest cells pair PhishStorm, PhiUSIIL or Phishpedia phishing with PhiUSIIL or crawled legitimate URLs.

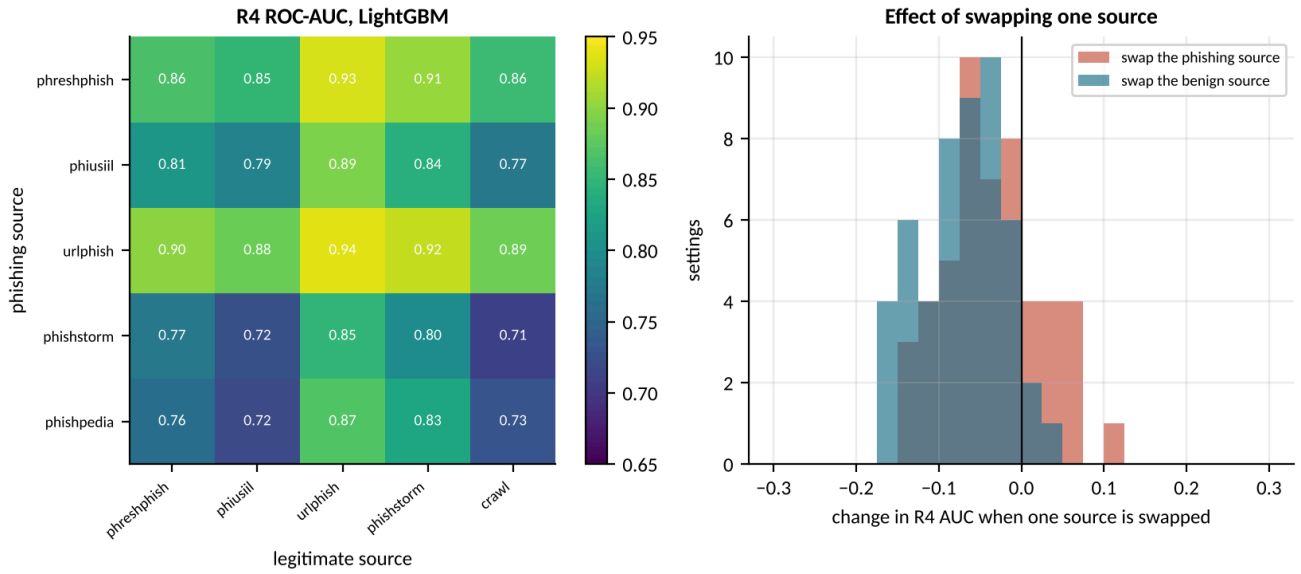


Figure 4.5. R4 AUC for every phishing-by-legitimate cell (left) and the distribution of swap effects (right).

the interaction check, an interaction between phishing and legitimate source, holds in 26% of settings; an interaction is hard to establish with this many settings, so it is treated as inconclusive.

4.4 Q4: split sensitivity (does not hold)

The median SD is 0.006 for R4 and 0.002 for R1, and the largest R4 SD is 0.019. R4 is the more variable in 100% of settings, but exceeds three times the raw SD in only 45% of them. With 5 to 10 thousand URLs per cell and domain-grouped splits the variability is small.

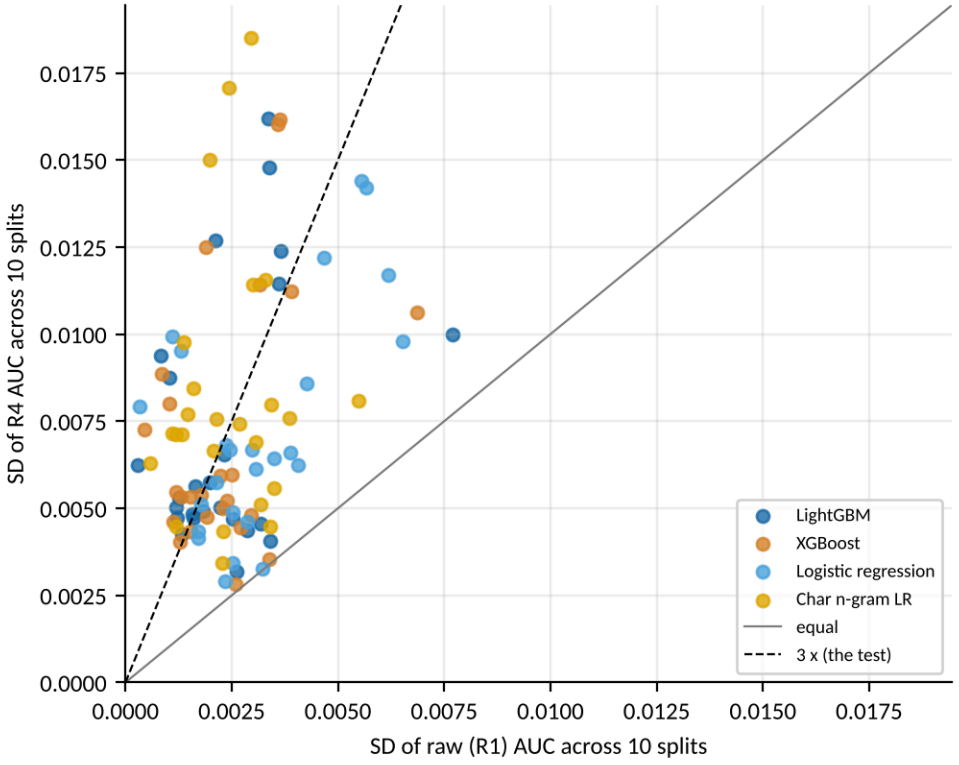


Figure 4.6. SD of AUC across ten splits, raw against R4, one point per cell and model.

4.5 Q5: platform masking (not testable)

Masking replaces hosting-platform domains with a neutral token. The caps used here leave no registrable domain on a hosting platform in any cell, so there is nothing to mask and the finding cannot be tested. A design that keeps platform domains is future work.

4.6 Q6: independent-benign transfer (holds)

Models trained on one cell and evaluated on cells that share no source with it keep an R4 AUC above 0.5 in every setting (median 0.699, lowest 0.554; 392 train-test pairs across the two transfer models). Each pair uses thousands of legitimate domains.

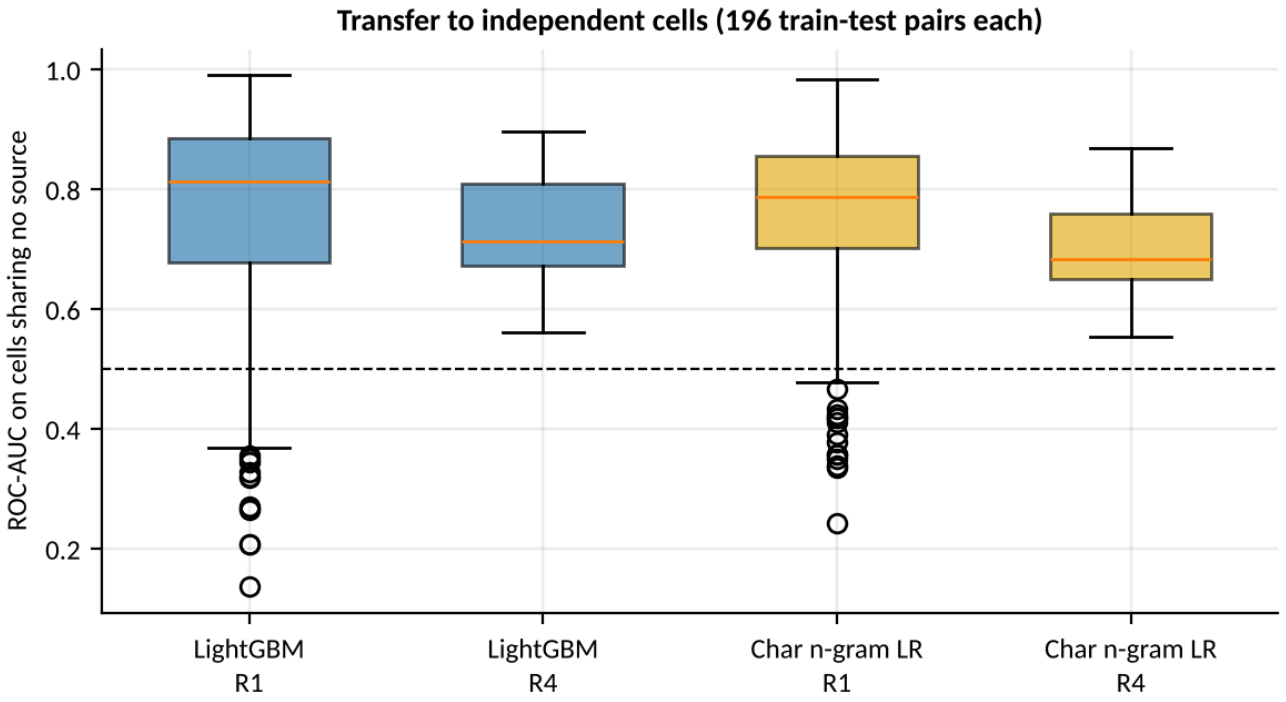


Figure 4.7. Transfer AUC on fully independent cells.

4.7 Q7: naturally browsed legitimate pages (holds in part)

A model's false-alarm rate on 2,555 natural legitimate pages from 562 sites, none in any training source, is compared with its own test side at the same 1% threshold. The median is 15.2% on natural pages against 1.0% in-cell; at the 0.1% threshold 4.5%; at the default 0.5 threshold 33%. The natural rate exceeds five times the in-cell rate in 76% of settings. No model flags every natural page, but a benchmark threshold still understates real-world false alarms.

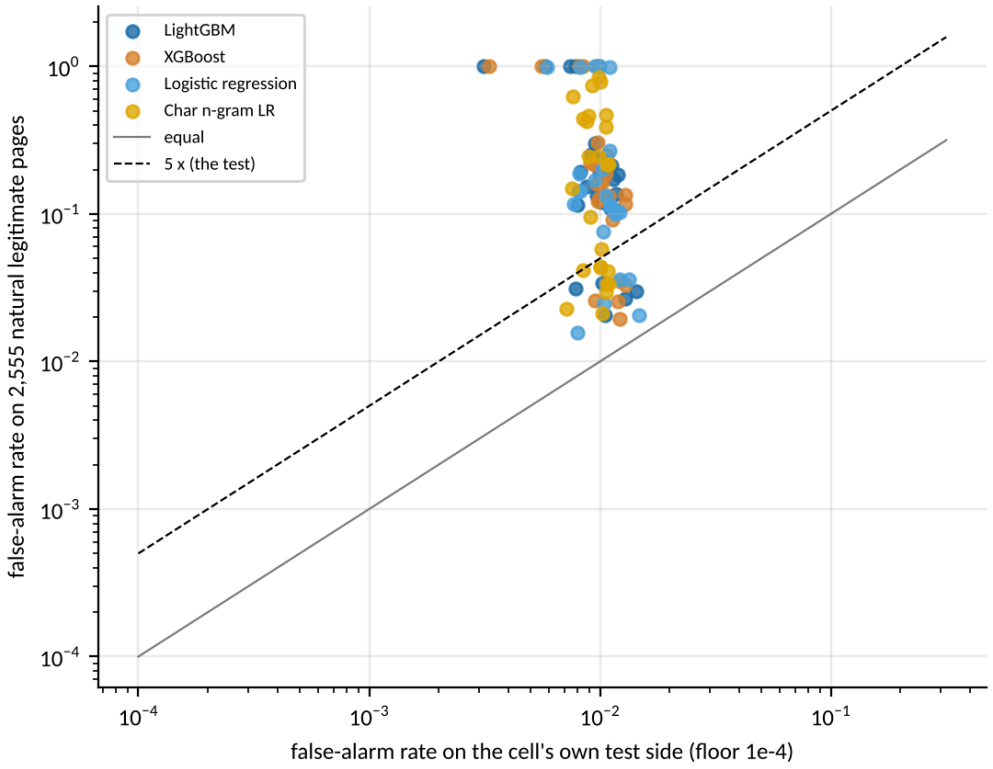


Figure 4.8. False alarms on natural pages against the cell's own test side, log scale.

4.8 Negative controls

With shuffled training labels, mean AUC stays between 0.448 and 0.525 over 100 conditions (cell, representation, model); 1 is flagged under the |mean - 0.5| > 0.05 rule. The flagged condition is examined in Section 4.10.

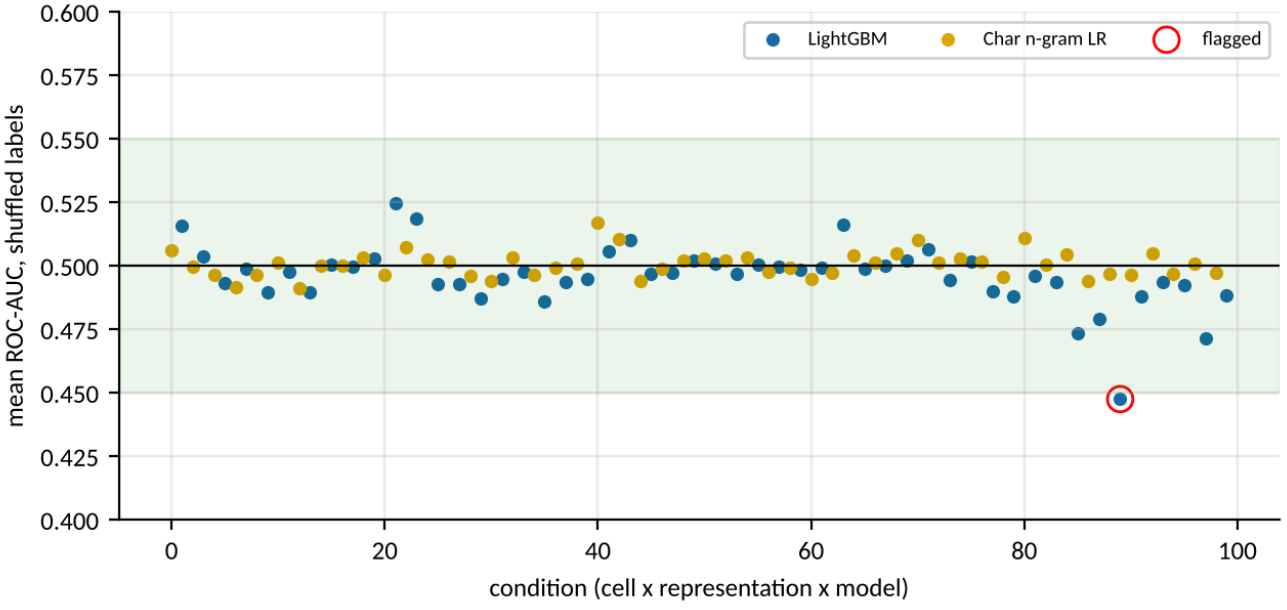


Figure 4.9. Shuffled-label controls, 100 conditions.

4.9 Dataset fingerprints

A model can tell which source a URL came from even within one class, which is the mechanism behind source dependence. The table gives balanced accuracy for predicting the source of legitimate and phishing URLs (chance 0.20 for URLs and 0.50 for page HTML). It falls when URLs are reduced to R4 but stays well above chance.

Table 4.3. Source identification within one class (5 domain folds).

part	class	input	balanced_accuracy	chance	n
A	benign	R1	0.621	0.200	44964
A	benign	R4	0.315	0.200	44964
A	phishing	R1	0.672	0.200	42885
A	phishing	R4	0.414	0.200	42885
B	benign	P1	0.820	0.500	14826
B	phishing	P1	0.935	0.500	13937

4.10 Follow-up checks

How data preparation changes the PhiUSIIL result. PhiUSIIL was re-run alone, adding one cleaning step at a time (5 splits). Raw PhiUSIIL through our loader scores about 0.94 (XGBoost); sampling to 10,000 per class, removing domains shared with other phishing sources, the pool rules and the study caps take it to 0.867, so data preparation alone moves the raw score by about 0.07. R4 stays between 0.79 and 0.85 at every stage.

The flagged shuffled-label control. In the URL-Phish phishing against PhiUSIIL legitimate cell, LightGBM trained on shuffled labels scores below chance at every seed: mean 0.449 (SD 0.031, 50 runs); the character n-gram model scores 0.506. A leak raises a control above 0.5, so this is read as a model artefact in one cell and listed as a limitation.

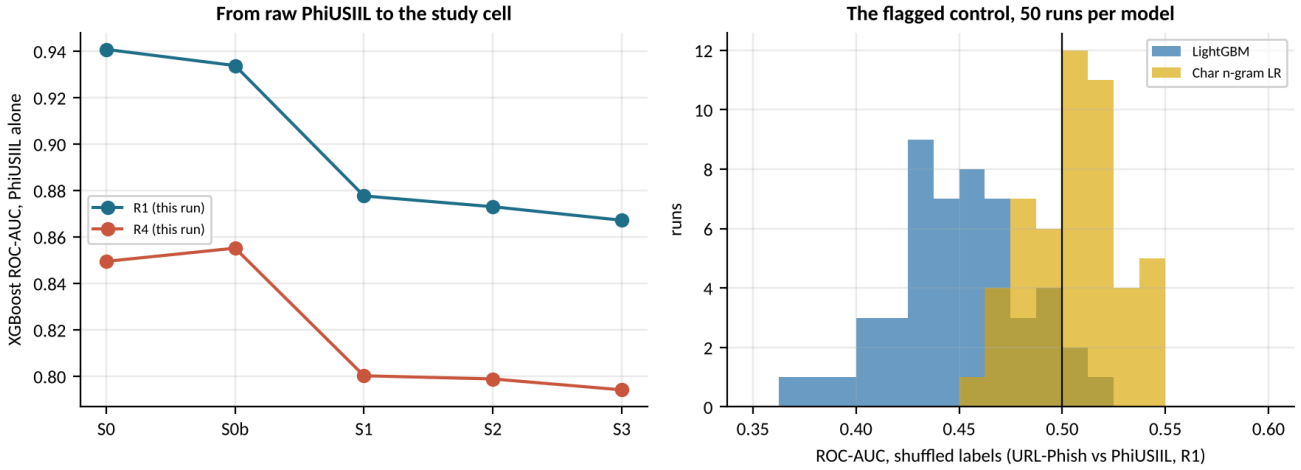


Figure 4.10. Left: XGBoost AUC from raw PhiUSIIL to the study cell. Right: the flagged control.

4.11 Page HTML (exploratory)

Four cells (Phishpedia or PhreshPhish phishing against PhreshPhish or crawled legitimate pages) use the 84 page features, grouped into P1 (all), P2 (scripts and structure removed), P3 (forms and text) and P4 (forms only). **Part B gives no verdicts.**

Table 4.5. Page and URL ROC-AUC, median of 10 splits.

cell	P1	P1+R1	P1+R4	P2	P3	P4	R1	R4
phishpedia vs crawl	0.997	0.999	0.998	0.997	0.992	0.903	0.966	0.740
phishpedia vs phreshphish	0.996	0.997	0.996	0.994	0.987	0.890	0.963	0.754
phreshphish vs crawl	0.988	0.996	0.987	0.982	0.955	0.837	0.981	0.857
phreshphish vs phreshphish	0.978	0.990	0.982	0.974	0.947	0.837	0.963	0.866

- **Page features do not show the URL drop.** P1 scores 0.978 to 0.997 where the R4 URL alone scores 0.740 to 0.866. Forms only (P4) still reaches 0.837 to 0.903.
- **Transfer is uneven.** A Phishpedia-vs-crawl model scores 0.687 on PhreshPhish cells (0.808 with the R4 URL added); the reverse direction scores 0.937.
- **Coverage is uneven.** Usable HTML exists for 91% of PhreshPhish benign rows but 63% of phishing rows; in PhreshPhish, *missing HTML alone* predicts phishing with ROC-AUC 0.639. Page numbers may therefore be optimistic.

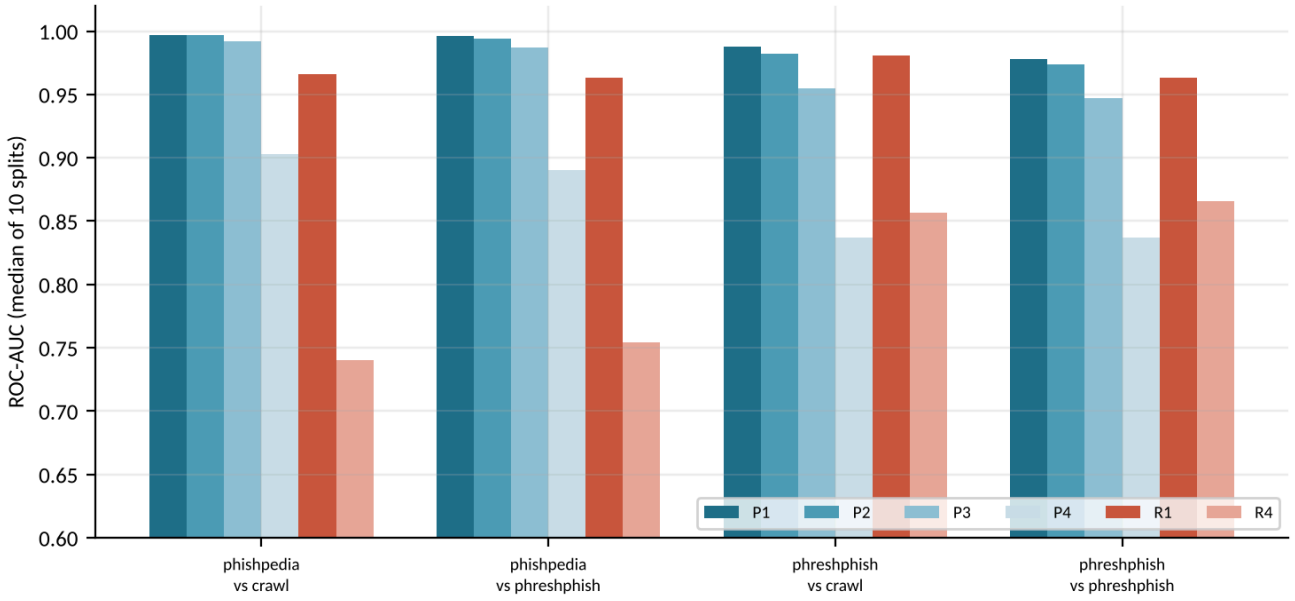


Figure 4.11. Page and URL representations, LightGBM.

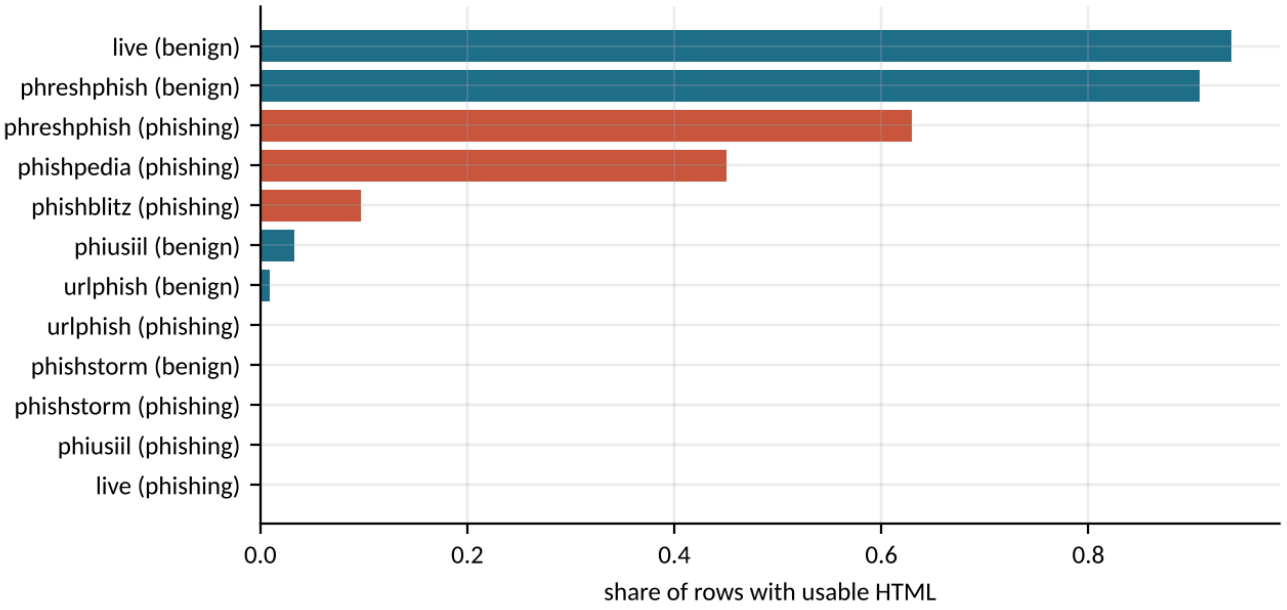


Figure 4.12. Share of rows with usable HTML, by source and class.

Table 4.6. False alarms on 2,226 natural legitimate pages with HTML.

cell	rep	n	fpr_0p5	fpr_0.01	fpr_0.001
phishpedia vs crawl	P1	2226	0.032	0.020	0.007
phishpedia vs crawl	P1+R1	2226	0.027	0.016	0.003
phishpedia vs phreshphish	P1	2226	0.027	0.023	0.004
phishpedia vs phreshphish	P1+R1	2226	0.059	0.062	0.009
phreshphish vs crawl	P1	2226	0.268	0.050	0.022
phreshphish vs crawl	P1+R1	2226	0.125	0.040	0.010
phreshphish vs phreshphish	P1	2226	0.201	0.061	0.010
phreshphish vs phreshphish	P1+R1	2226	0.193	0.102	0.043

4.12 Limitations of the study

- **Sources.** Two sources (PhiUSIIL, PhishStorm) are homepage-heavy or old; PhishTank is not in the pool. Results describe these public datasets.
- **Not testable.** Q5, and per-source temporal effects: PhiUSIIL carries no per-row dates.
- **Sample size.** Cells use up to 10,000 URLs per class with 5 per domain; this lowers cross-split variability compared with a larger pool.
- **Feature engine.** R3 equals R2 by construction, because the engine drops a leading `www`.
- **Control.** One of 100 shuffled-label controls is flagged (below chance, one cell).
- **Part B.** Four cells, two sources with saved HTML, uneven coverage by class.
- **No hand-typed results.** Every number is read from `reports/phishguard/tables/` by `python -m phishguard.report`; seeds are fixed.

4.13 A second condition: data prepared without the cleaning

Questions 2 and 4 and the interaction check did not hold in the main design. The main design cleans its data: it caps URLs per registrable domain, removes domains shared by several phishing sources and balances URL depth. Before any result was seen, a second condition was written down in `docs/PHISHGUARD_SENSITIVITY_PLAN.md` that rebuilds the same populations *without* that cleaning (only exact duplicates and URLs labelled both ways are removed; each population is sampled to at most 10,000 URLs per class). Models, splits, bootstrap, the rule for "holds" and its cut-offs are unchanged. The crawled legitimate population cannot be rebuilt without the cleaning, so both conditions are compared on the same 20 cells (five phishing sources by four legitimate sources). The comparison was run once; nothing was changed after seeing it.

Question	Cleaned data (same 20 cells)	Without the cleaning
Q1	holds (80 of 80, 100%)	holds (79 of 80, 99%)
Q2	does not hold (0 of 80, 0%)	does not hold (0 of 80, 0%)
Q3	holds (232 of 280, 83%)	does not hold (27 of 280, 10%)
Q4	does not hold (31 of 80, 39%)	holds in part (55 of 80, 69%)
interaction	does not hold (78 of 240, 32%)	does not hold (0 of 240, 0%)

The median score of the 20 cells is 0.976 on raw URLs and 0.857 on domain-only URLs with the cleaning, against 0.982 and 0.862 without it (the table below lists every cell). Source: `reports/phishguard/sensitivity_uncleaned.md`.

What this shows.

- Question 1 holds in both conditions.
- Question 2 (how much of the fall sits in the host name) does not hold in either, so it is not explained by the cleaning.
- Question 4 (the spread of the domain-only score across splits) does not hold with the cleaning and holds in part without it, so it depends on how the data is prepared.
- Question 3 (source dependence) is the reverse: it holds with the cleaning (232 of 280 settings in these 20 cells) and does not hold without it (27 of 280). The cleaning is part of what makes the source effect show up, so Question 3 is a result of the cleaned design and should be reported that way.
- The interaction check does not hold in either condition.

The honest summary is that the answers to Questions 3 and 4 depend on how the benchmark is prepared, in opposite directions. Neither condition is the "true" one; both are reported.

Cell	Cleaned R1	Cleaned R4	Uncleaned R1	Uncleaned R4
phishpedia__phishstorm	0.975	0.831	0.981	0.886
phishpedia__phiusiil	0.998	0.727	0.996	0.773
phishpedia__phreshphish	0.968	0.769	0.970	0.837
phishpedia__urlphish	0.993	0.872	0.994	0.893
phishstorm__phishstorm	0.972	0.798	0.968	0.821
phishstorm__phiusiil	0.995	0.725	0.995	0.742
phishstorm__phreshphish	0.978	0.776	0.976	0.801
phishstorm__urlphish	0.991	0.854	0.991	0.853
phiusiil__phishstorm	0.988	0.841	0.990	0.899
phiusiil__phiusiil	0.860	0.792	0.931	0.868
phiusiil__phreshphish	0.948	0.812	0.972	0.861
phiusiil__urlphish	0.923	0.895	0.959	0.927
phreshphish__phishstorm	0.983	0.907	0.985	0.905
phreshphish__phiusiil	0.958	0.860	0.984	0.885
phreshphish__phreshphish	0.955	0.870	0.956	0.852
phreshphish__urlphish	0.970	0.935	0.986	0.941
urlphish__phishstorm	0.977	0.923	0.970	0.829
urlphish__phiusiil	0.985	0.886	0.990	0.863
urlphish__phreshphish	0.967	0.903	0.955	0.737
urlphish__urlphish	0.980	0.939	0.983	0.906

Median ROC-AUC per cell on raw (R1) and domain-only (R4) URLs in both conditions.

4.14 Hosting-platform masking (exploratory)

Question 5 asks whether replacing the name of a hosting platform (for example `github.io`) with a neutral token helps a model on tenants it has not seen. It was not testable in the main design, because the cap per domain and the removal of shared domains leave almost no platform URLs. An exploratory test was therefore run outside the seven questions: platform URLs were taken straight from the source files, at most 5 URLs per tenant site were kept, splits were grouped by registrable domain, and a model trained with the platform name masked was compared with one trained without masking on domain-only URLs. Only one legitimate source (PhreshPhish) has enough platform URLs (1,019 URLs on 617 tenant sites), so the five tested pairs share the same legitimate set and are not independent.

Masking raised the held-out score in none of the 10 settings (five phishing sources, two models, ten splits each). The median change ranges from -0.014 to +0.012 and every interval includes zero. The effect of masking is therefore **not shown** in this test. This does not show that masking never helps; it shows that no effect could be detected here. Source: `reports/phishguard/HP_platform.md`.

4.15 What can and cannot be claimed

Table 4.7. Claim register: each finding with the wording this run supports.

Finding	This run	Verdict	How to word it
Q1 Representation sensitivity	R1 0.977 to R4 0.850 (median of cells and models); falls in every one of 100 model-cell settings	Holds	A direct finding, across five phishing sets, five legitimate sets and four model families.
Q2 Where the drop sits	median host/subdomain share 0.21 on the study cells; on raw PhiUSIIL alone 0.76 (XGBoost), falling to 0.36 after cleaning	Does not hold	Say the host/subdomain share depends on how the benchmark is prepared: it is high on the raw benchmark and low on cleaned pools.
Q3 Source dependence	swapping one source moves R4 AUC by a median 0.058; CI excludes 0 in 76% of settings, 53% after depth reweighting	Holds in part	Real in some pairs, absent in others; reweighting for depth halves it.
Q4 Split sensitivity	median SD 0.006 (R4) against 0.002 (R1); larger than 3x in 45% of settings, largest 0.019	Does not hold	Split variability matters but the 3x margin is not general.
Q5 Platform masking	no hosting-platform domains remain after the 5-per-domain cap	Not testable	Leave to a design that keeps platform domains.
Q6 Independent transfer	R4 AUC 0.699 median, lowest 0.554, above 0.5 in every setting	Holds	Transfer stays above chance; the size varies.
Q7 Natural pages	15.2% flagged on natural pages at the cell's 1% threshold against 1.0% in-cell; 33% at the 0.5 threshold	Holds in part	Benchmark thresholds understate false alarms on natural pages.

5. Data and corpus

Scope. These chapters describe PhishDrift, the software: a layered phishing detector shipped as a Chrome extension (Manifest V3) with a FastAPI backend. The evaluation study (Chapters 3 and 4) is not repeated here.

How to read the numbers. Every number is copied from a generated file under `reports/` or from `docs/PROGRESS.md`, and the source is named in brackets. Unless stated otherwise, a result is the median over 10 domain-grouped splits, with [min, max] in square brackets, measured with the allowlist and blocklist layers switched off. “Recall at FPR 1%” and “recall at FPR 0.1%” are the headline metrics. “Operating point” (op) means the largest recall whose false-positive rate on that same set is at or below the target. “Cal” means the threshold was chosen on the calibration side and then applied unchanged, which is what a deployed model does. Accuracy is never reported as a headline. Items I could not verify are marked “not measured”.

Corpus counts in chapter 5 come from the corpus data card as quoted in the 2026-10-13 report draft. The data card file itself is not in this checkout, so those counts are carried over and were not re-checked against it.

5.1 Why the corpus is built this way

The evaluation study (Chapter 4) shows that a benchmark score depends on how URLs are written, which sources make up each class and which split is used, and that a threshold set on a benchmark understates false alarms on ordinary pages (`docs/PROJECT_SPEC.md`). PhishDrift therefore treats the data pipeline as part of the product. The rules that follow are fixed in `CLAUDE.md`: only URL, label, date and HTML are read from any dataset, never a precomputed feature column; one registrable-domain definition is used everywhere (public-suffix-only); and no model is trained until the corpus passes a depth check.

5.2 Sources

All sources are loaded into one schema: `url, domain, label, date, source, html, depth`. Table 5.1 shows the training pool, the newest time slice and the held-out source.

source	part	URLs	domains	phishing	benign	with HTML	dates
PhreshPhish	pool	147,232	106,917	46,602	100,630	147,232	2024-07-02 to 2025-09-08
PhreshPhish	time slice	80,653	58,054	24,289	56,364	80,653	2025-09-08 to 2025-12-16
PhiUSIIL	pool	171,826	157,550	52,452	119,374	4,036	load date only
URL-Phish	pool	91,041	84,130	4,367	86,674	830	load date only
Phishpedia	pool	9,580	5,897	9,580	0	9,254	2019-06-22 to 2020-09-30
Phish-Blitz	pool	248	209	248	0	92	2025-09-24
Live feeds	pool	245	97	133	112	112	2026-10-10
PhishStorm	held-out source	59,485	33,056	36,293	23,192	0	none (2014)

Table 5.1. Sources by part (corpus data card, as quoted in the 2026-10-13 draft; not re-verified).

Three points need care. First, PhiUSIIL and URL-Phish carry no per-row dates (the data card shows only the load date), so they are never used for the time slice. Their HTML counts are crawled inner pages, which belong to the source that seeded the crawl. Second, Phish360, the planned held-out source, has no public download, so PhishStorm (URL only, both classes, 2014) was held out instead (`docs/PROGRESS.md`, 2026-10-11). Because PhishStorm has no HTML, the page model has no held-out source at all. Third, the live feeds are still tiny in the pool (245 URLs); they matter mainly for tier C (section 6.4). Licences recorded in the data card: PhreshPhish CC BY 4.0 (anti-phishing research only), PhiUSIIL CC BY 4.0, Phishpedia research use, OpenPhish non-commercial, Phishing.Database MIT; the URL-Phish and Phish-Blitz terms are still to be confirmed.

5.3 Corpus build

The corpus is built before any split, in this order (`docs/PROJECT_SPEC.md`, "Corpus build"): load each source into the schema; canonicalise every URL the same way and deduplicate across all sources, dropping any URL with conflicting labels; remove near-duplicate HTML within each label; remove every registrable domain that appears in more than one phishing source from all sources; keep each domain in one source with a cap on URLs per domain; mix homepage-only benign sources with deep benign URLs; check per-class depth; carve out the held-out source and the newest time slice.

The data card reports 1,151,098 rows loaded and 1,200 registrable domains that appeared in more than one phishing source. The shared-domain rule removed the most rows in several sources (111,203 from PhreshPhish, 40,451 from PhiUSIIL), consistent with hosting platforms and shorteners dominating the overlap between phishing corpora. PhreshPhish also lost 162,433 rows to near-duplicate HTML removal, which explains why more rows were loaded from it than its published size (data card, "Rows removed per step and source").

part	URLs	domains	phishing	benign	phishing %	with HTML
pool	420,172	354,800	113,382	306,790	26.98	161,556
time slice	80,653	58,054	24,289	56,364	30.12	80,653
held-out source	59,485	33,056	36,293	23,192	61.01	0

Table 5.2. Corpus parts (data card; URL and domain counts are kept separate because URLs per domain differ a lot between sources).

The pool is then split 10 times into train, calibration and test sides, grouped by registrable domain. The median test side holds 84,036 URLs from 71,044 domains, of which 22,678 are phishing and 61,364 benign (`reports/tier_a.md`). Zero domain overlap and zero canonical-URL overlap between sides is a hard run-time failure, not a warning.

5.4 Depth balance

PhiUSIIL's legitimate URLs are all bare homepages, so a model trained on them alone learns "has a path means phishing". The build therefore reports the per-class depth distribution and stops if any depth bucket is more than 90% one class. On the training pool the check passed; the most one-sided bucket was depth 2, with 78.32% of its rows in one class (data card, "Depth check on the training pool"). Inside single sources the shortcut is still present: 96.62% of PhiUSIIL benign rows are depth 0, depth 0 is 97.37% one class in URL-Phish, and in the held-out PhishStorm depth 0 is 96.57% one class and depth 4+ is 90.87% one class (data card, "Depth by source"). This is why chapter 8 adds a depth-matched evaluation and a representation check instead of trusting pooled numbers.

5.5 Benign inner pages

Deep benign URLs come from PhreshPhish benign (real browsing telemetry) and from a crawler that fetches inner pages of seed sites, honouring robots.txt and Crawl-delay. Of 1,710 PhiUSIIL seed domains tried, 954 gave pages (4,819 pages,

55.8%); of 400 URL-Phish seeds, 201 (1,008 pages, 50.2%); of 60 live seeds, 23 (147 pages, 38.3%) (data card, “Benign inner-page crawl”). About half of the seed sites are therefore unreachable or crawl-hostile, so the crawled benign pool is biased towards sites that are easy to crawl, a bias the prior report also saw (section 8, 16.1 of that report).

A larger “popular sites” crawl (Chrome’s most visited hosts, plus Common Crawl index URLs) was added as code on 2026-10-20 to give the model more deep legitimate URLs. Its first 20-site test on the project PC returned no pages from the Common Crawl index, and a fix was written but not yet run (`docs/PROGRESS.md`, 2026-10-20). The corpus described above does not include it, and its effect on false alarms is not measured.

5.6 Evaluation-only data

Two legitimate sets are never trained on. The first is a set of 2,555 ordinary legitimate inner pages from 562 sites that are in no training, calibration or held-out set (the “natural pages”); a second crawl pass kept HTML for 2,226 pages from 512 sites (`reports/blend_ood.md`). It is used only to measure false alarms. The second is a set of 104 to 114 regional brand sites (Indian, European and Asian banks, payment apps and login pages) used for the look-alike rules and the URL-layer test (`reports/brand_rules.md`, `docs/PROGRESS.md`). The natural pages were sampled from Tranco-listed sites, which makes them easier than a long-tail browsing history; this is a limitation of every false-alarm number below.

6. Feature engine and models

6.1 One engine, three tiers

All features come from one engine, `extract(url, html, domain_info)`, with missing values as explicit NaN. Every feature is registered once in `features/registry.py`, `docs/FEATURE_DICTIONARY.md` is generated from the registry, and a test fails if the document is stale. The tiers exist because missingness would otherwise reveal the source: a single model with NaN for "no HTML" could learn which dataset a row came from.

tier	input	features	available for	where it runs
A	URL	99	every URL	browser
B	URL + page HTML	84 added	URLs with usable HTML	browser (content script)
C	domain, DNS, TLS captured at first sight	29	live-feed hosts only	backend (offline experiment)

Table 6.1. Feature tiers (`reports/hybrid2.md`, `reports/tier_b.md`, `reports/tier_c.md`).

Two rules protect the models from leakage. Popularity signals (Tranco rank, traffic, search indexing) and blocklist membership are never model inputs; rank is used only to cut the allowlist. Registrable domains of hosting platforms (from the public suffix list's private section, `configs/platforms.yaml`) are replaced by a neutral token before URL features are computed, and only `is_hosting_platform` is exposed, so that platform identity cannot stand in for phishing (the effect of masking was not testable in Chapter 4: no platform domains remain in the study cells). Tier A features are ported to JavaScript and checked against Python on 272 fixed URLs in both test suites (`docs/PROGRESS.md`).

A source-leakage check compared, for each tier A feature, how strongly it separates sources within a class against how strongly it separates the classes. 17 of 99 features were flagged, mostly path and length features (`n_slash`, `path_depth`, `entropy_url`, `url_len`). Dropping the 17 lowered held-out LightGBM ROC-AUC from 0.9168 to 0.8943 and recall at 1% FPR from 59.1 to 55.3 (`reports/tier_a.md`), so they were kept and the question is reported rather than hidden. None of the 84 tier B features was flagged (draft, from `reports/tables/feature_source_check_b.csv`).

6.2 Tier A: from three base learners to the hybrid

The three base learners are LightGBM and XGBoost on the 99 features and a character n-gram TF-IDF with logistic regression whose vocabulary is fitted on training folds only. On the pooled test side LightGBM reaches ROC-AUC 0.9182 and 57.8% recall at FPR 1%; the char n-gram model reaches 0.9363 and 61.3% (`reports/tier_a.md`).

An error analysis on one split showed that 70% of missed phishing URLs were bare domains with no path, mostly from PhiUSIIL (recall at FPR 1%: PhiUSIIL 47%, PhreshPhish 66%, URL-Phish 79%; `docs/PROGRESS.md`, 2026-10-12). A model with only a domain name to judge has a ceiling. Several combinations were tried on one split, chosen on its calibration slice only; the best was a LightGBM that receives out-of-fold char n-gram scores as extra features (68.4 / 50.1 against 58.2 / 40.0 for plain LightGBM). The model that ships is "hybrid2": the 99 features plus four out-of-fold text scores (URL, host, path as character models, and a word model on the URL). A stacked alternative (LightGBM + XGBoost + char n-gram, logistic-regression meta model, isotonic calibration) was also built; it is better calibrated (Brier 0.0681, ECE 0.0039 on test) but not better at ranking (`reports/stack_a.md`). The hybrid is a single model, which made it the one to export to the browser (`docs/PROGRESS.md`).

6.3 Tier B: the page model and the blend

The tier B page model is a LightGBM on the tier A features, the hybrid2 text scores and 84 page features: credential forms and fields, link and resource ratios, title and copyright matches against the domain, page structure and script obfuscation. Its largest gain shares are `tld_risk` (33.0% of top-25 gain), `url_title_match` (10.8%), `n_external_link_domains` (9.6%) and `link_internal_ratio` (9.1%) (`reports/tier_b.md`). It is exported to a 2.46 MB JSON file and runs in the extension's service worker (`docs/PROGRESS.md`, 2026-10-14).

The "blend" routes each URL to the model the browser would use: the page model where the page has usable HTML (at least 200 characters of visible text and not a parked or error page), and hybrid2 otherwise. Isotonic calibration on the calibration side puts both on one scale, and one pair of thresholds is taken from the pooled blended calibration scores (`reports/blend2.md`).

6.4 Tier C: domain, DNS and TLS

Tier C features (registration age and expiry from RDAP, DNS records, certificate age, validity and issuer) can only be computed honestly when captured as soon as a URL is first seen, so the collector records them for live-feed hosts, and for legitimate Tranco hosts, twice a day. They are never computed today for old dataset URLs. Tier C is therefore trained on live data only, and its sample is small (section 8.9). A separate set of hand-set backend rules (section 7.5) uses the same facts at run time; it is not the tier C model.

6.5 Calibration and thresholds

Thresholds are never 0.5. "Suspicious" is the threshold at FPR 1% and "dangerous" the threshold at FPR 0.1%, both chosen on each split's calibration side, and isotonic calibration maps scores to probabilities. Fixed-FPR points use the largest attainable recall with achieved FPR at or below the target, and a 0.1% point resting on fewer than 10 negatives is marked granularity-limited. Calibration is good where the data match: ECE 0.0039 on test, but 0.0369 on the held-out source (`reports/stack_a.md`). Chapter 8 shows that the thresholds themselves do not transfer well to a new source.

7. System: extension and backend

7.1 Runtime layers

The extension checks each navigation with layers ordered from cheapest to most expensive (Figure 7.1). The order is fixed in `CLAUDE.md` and implemented in `extension/src/background.js` and `extension/src/combine.js`.

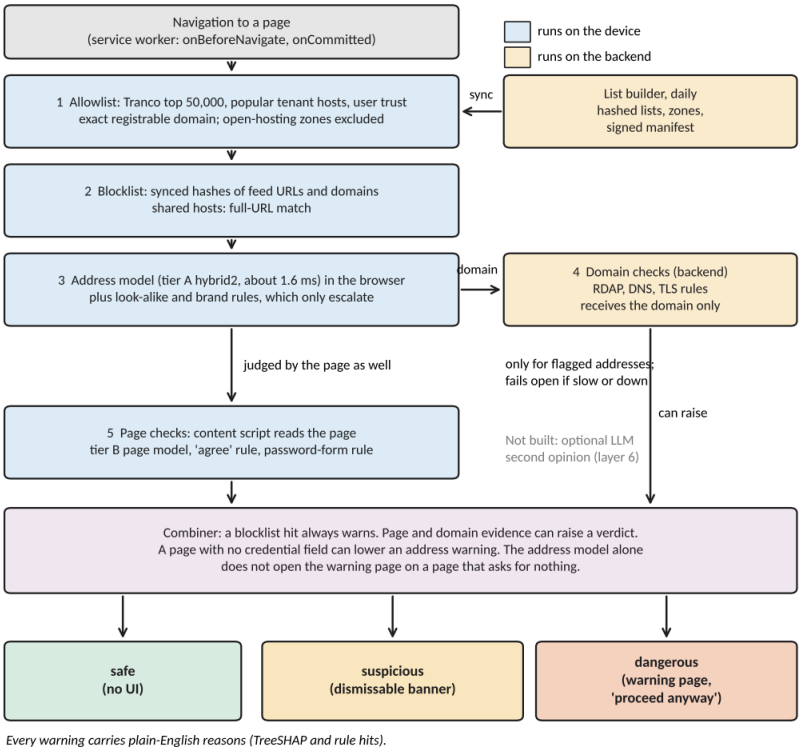


Figure 7.1. Runtime layers of PhishDrift and where each runs. No data; drawn by scripts/pbl_figures.py.

1. **Allowlist.** Tranco top 50,000, a list of popular tenant sites, and user-trusted sites, matched on the exact registrable domain. Shared-hosting domains and learned open-hosting zones are excluded. The bundled list is used until a synced list exists.
2. **Blocklist.** Hashed URLs and domains built by the backend from the live feeds and URLhaus. On shared hosts the full URL is matched, not the domain.
3. **Address model.** The tier A hybrid2 model, in the browser, followed by look-alike and brand rules that can only raise a verdict.
4. **Domain checks.** The backend runs RDAP, DNS and TLS checks for the domain, only for addresses the model already flagged.
5. **Page checks.** The content script sends the page to the service worker, where the tier B page model and a password-form rule read it.

A combiner turns the layers' outputs into safe (no interface), suspicious (a dismissable banner) or dangerous (a warning page with "proceed anyway" and "report false positive"), each with plain-English reasons from TreeSHAP and rule hits. The optional language-model second opinion (layer 6 in the specification) was not built; it was first on the cut list.

7.2 How a verdict is decided

The rules in `extension/src/combine.js` and the progress log are these.

- A blocklist hit always warns. Neither "trust this site" nor the allowlist overrides an exact listed URL; a user's trust entry expires after 30 days.
- The page model may raise any address verdict. It may lower one only on a page with no password, one-time-code, card or PIN field in the page or any of its frames, because on a login page the attacker controls the markup that the page model reads.
- The "agree" rule: on a page with usable HTML, a suspicious banner needs both the page model and the address model to exceed their 1% thresholds. Its effect is measured in section 8.5.
- The address model alone does not open the warning page on a page that asks for nothing. A dangerous address verdict is shown as a banner unless the page asks for a credential, the page model also says dangerous, a second layer raises it, or no page check arrives within 5 seconds (extension 0.1.2, `reports/address_only_blocks.md`).
- Domain evidence from the backend can raise suspicious to dangerous but never turns a safe verdict into a warning. The backend is never waited for: the verdict shows at once and a late "high" answer raises it.
- Short links are never allowlisted and never judged by the address model; the redirect target is scored at `onCommitted`, before the page renders. A shortener's own address carries no information for a URL model (section 8.10).
- A verdict cannot be lowered by the page itself: `history.replaceState` and later route changes keep the worst verdict per tab, login forms added after load are re-checked up to three times, frames are checked in their own right, and a page that hides or removes the banner gets the full warning page.
- Pages with no web address of their own (data URLs, sandboxed frames) are not scored and warn only if they ask for credentials.

7.3 Lists, zones and sync

The allowlist is the layer that keeps the extension quiet on sites people use: without it the address model alone flags 16.1% of the Tranco top 1,000 homepages (`reports/shipped_verdicts.md`). It also had to be made safer. A phishing page under `*.swedencentral.cloudapp.azure.com` passed as `azure.com`, so the list builder learns open-hosting zones from phishing data: each phishing host below an allowlisted domain marks the child zone that contains it (`cloudapp.azure.com`, `drive.google.com`), and phishing spread over many tenants marks the whole domain (`weebly.com`). Inside a zone the allowlist does not apply and bad URLs are listed one by one. After the zone evidence was extended with the 789k-URL Phishing.Database active list, 174 allowlisted domains became whole zones, while `google.com`, `microsoft.com`, `apple.com`, `paypal.com` and `github.com` stayed allowlisted (`docs/PROGRESS.md`, 2026-10-14). The zone evidence is never blocklisted and never trained on.

The reverse problem was a popular tenant site on a platform domain: `stitch.withgoogle.com` was flagged because `withgoogle.com` is on the platform list. A tenant that is itself popular (Chrome's top origins, rank at most 50,000) is now allowlisted by exact host unless phishing was seen on it, it is a zone or it is a shortener. Popular tenant homepages flagged fell from 60 of 137 to 0, phishing on platform hosts warned stayed at 2,740 of 3,000, and natural legitimate pages flagged stayed at 72 of 2,555 (`reports/tenant_sites.md`). A popular tenant that is abused tomorrow stays trusted until the next list build.

The backend builds the hashed lists and the extension syncs them every 60 minutes, with a signed manifest, file hashes, sanity limits on size changes, and deltas. Blocklist keys follow Safe-Browsing-style URL expressions, so a phishing URL on a shared host still matches when a query or token is appended. On the test thread's variants, matches rose from 0 of 300 to 298 of 300 (query added), 0 of 30 to 24 of 25 (value changed), 0 of 189 to 158 of 184 (last token changed) and 0 of 300 to 297 of 300 (token appended) (`docs/PROGRESS.md`, 2026-10-19; this is a measurement on constructed variants).

Signatures are off until a key is generated on the backend machine, and the bundled allowlist held 43,595 domains at the 2026-10-19 rebuild (137 tenant hosts were added later) because the cloud rebuild could not download Tranco; the PC rebuild restores 5,740 more (same entry).

7.4 Brand and look-alike rules

A small rule layer in Python and JavaScript (`features/brand_rules.py`, `js/src/brand_rules.js`, parity-tested on 175 URLs and 18 pages) only raises a verdict. Address rules look for a Unicode homograph skeleton of a brand, an embedded real domain, a name one letter from a brand, and brand plus a lure word; a page rule fires when a page names a brand in its title, site name or copyright, asks for a password, code, card or PIN, and is not on the brand's domain. The brand list has 899 names (`docs/PROGRESS.md`, 2026-10-20).

On the natural pages the address rules fire on 0 of 2,373 URLs (572 domains) and on 0 of 114 regional brand sites, and the page rule fires on 1 of 2,373 pages (`cashinexch.com/redirect/facebook`). On 300 OpenPhish URLs the address rules fire on 13 (4.33%) (`reports/brand_rules.md`). The rules were tuned after the natural set was seen, so these false-alarm counts are not independent. TLD swaps such as a bank's name on another suffix are not caught, because brands own many suffixes.

7.5 Backend and privacy

The FastAPI backend serves `/health`, `/score`, `/domain-info`, `/register`, `/feedback`, `/config` and the list endpoints, with a per-domain cache, rate limits and timeouts (`docs/BACKEND.md`). A failed or timed-out lookup returns `risk: "unknown"` with an error field, never "low", and the extension then fails open with the model's own verdict. The domain rules are hand-set thresholds, not the tier C model, and `/score` returns `"tier_c_model": false`.

Privacy follows from the design. Scoring happens on the device. The backend accepts a bare hostname only and rejects any other field, so a full URL or page content cannot be sent by accident. Feedback stores the domain, kind of report, verdict, a timestamp and a one-way per-install key; no IP address or user agent, and no browsing history is stored. The extension requests the permissions `storage`, `alarms` and `webNavigation` and host access only to the local backend address (`extension/manifest.json`). Feedback needs a per-install token (without one the request now gets 401, where it got 200 before) and is capped per install per day.

On the first-seen captures, the backend's own rule code was run on 140 phishing hosts (103 domains) and 1,398 legitimate hosts (770 domains). It returned risk "high" for 9.3% of phishing hosts and 0.0% of legitimate ones, and "elevated" for 3.6% and 0.1% (`reports/domain_rules.md`). Signals that fire as often on legitimate hosts (unregistered domain 22.1% against 22.0%) were demoted to "info". This is a small sample, and the severities are set from it.

7.6 Latency

Measured in Node (V8, the engine Chrome uses) on a 4-CPU machine over the 272 parity URLs, 5 runs each (`reports/latency.md`).

step	median	95th percentile	max
score one URL (features, text models, trees)	1.6 ms	2.3 ms	6.5 ms
score one URL and explain it (TreeSHAP reasons)	156.5 ms	196.1 ms	277.7 ms

Table 7.1. Latency of the in-browser address model.

Start-up, once per browser session, takes 166 ms to parse the 11.7 MB model bundle and 73 ms to build the model. The extension scores every navigation without reasons and asks for reasons only when a warning is shown. Page-model latency and backend response times are not measured.

7.7 URL check page and user interface

`python -m demo` serves a local page where a pasted address is walked through the allowlist, blocklist, address model, look-alike rules and combiner, one screen per layer, ending in the verdict, score against both thresholds and reasons (`docs/PROGRESS.md`, 2026-10-20). It does not run the page model, the password-form rule or the backend.

Screenshots are not available in this environment, so the interface is described in words. A suspicious verdict shows a small banner, in a closed shadow root at the top of the page, with the reasons and buttons to dismiss, trust the site or report a false alarm. A dangerous verdict replaces the page with a warning page that lists the reasons, offers “Take me back to safety”, “Proceed anyway” and “Report false positive” (which sends only the domain). The toolbar popup shows the current verdict and its reasons. Screenshots of each state should be taken on the author’s own browser for the final report.

7.8 Screenshots

The environment that built the system cannot browse, so these slots are for the author’s own screenshots. Each slot says what the picture should show.

Screenshot 1. A page judged safe. Paste screenshot here
Should show: a normal site open in Chrome with the PhishDrift icon showing no warning.

Screenshot 2. Suspicious banner. Paste screenshot here
Should show: a page with the yellow banner at the top and its one-line reason.

Screenshot 3. Dangerous warning page. Paste screenshot here

Should show: the full warning page with the plain-English reasons and the buttons.

Screenshot 4. Popup with reasons. Paste screenshot here

Should show: the toolbar popup open on a flagged page, listing the reasons.

Screenshot 5. Report a false alarm. Paste screenshot here

Should show: the button or form used to report a wrong warning (domain only is sent).

Screenshot 6. URL check page, layer view. Paste screenshot here

Should show: one layer screen of the URL check page, for example the address-model gauge.

Screenshot 7. URL check page, verdict. Paste screenshot here

Should show: the final verdict screen with the shield, reasons and score meter.

8. Evaluation of the detector

8.1 Protocol

Every split is grouped by registrable domain, and the split code stops with an error on any domain or canonical-URL overlap. Results are medians over 10 splits with their spread, never one split. Allowlist and blocklist layers are disabled, because the feeds supply the test URLs. The test side holds a median of 84,036 URLs (61,364 benign), the held-out source 59,485 (23,192 benign) and the time slice 80,653 (56,364 benign) (`reports/summary.md`), so a 0.1% point allows about 61, 23 and 56 false positives respectively, and in per-bucket views fewer than ten (marked with an asterisk where it applies). No held-out-source result above 99% appears anywhere below, so the leakage tripwire did not fire there; the one place above 0.99 on the test side (tier B on HTML rows) was investigated separately (section 8.4).

8.2 Main results: test side, held-out source, time slice

model	ROC-AUC	recall @1% FPR	recall @0.1% FPR	cal 1%: recall / FPR
LightGBM (A)	0.9182	57.8 [57.1, 58.9]	39.9 [38.5, 42.0]	57.7 / 1.0
char n-gram + LR (A)	0.9363	61.3 [60.0, 61.9]	40.7 [37.8, 43.3]	61.1 / 1.0
stack (A)	0.9490	66.5 [65.1, 67.4]	46.3 [44.5, 47.8]	66.3 / 1.0
hybrid2 (A, shipped)	0.9536	69.5 [68.7, 70.9]	52.1 [49.2, 54.0]	69.8 / 1.0
blend (A + B, shipped)	0.9615	74.6 [73.5, 75.6]	59.6 [57.8, 60.9]	74.7 / 1.0

Table 8.1. Pooled test side, 84,036 URLs from 71,044 domains (`reports/tier_a.md`, `reports/stack_a.md`, `reports/hybrid2.md`, `reports/blend2.md`).

set	model	ROC-AUC	recall @1% FPR	recall @0.1% FPR	cal 1%: recall / FPR
held-out PhishStorm	LightGBM	0.9168	59.1 [58.0, 59.5]	40.6 [37.5, 41.5]	67.8 / 2.5
held-out PhishStorm	char n-gram + LR	0.9211	54.9 [54.2, 56.0]	32.3 [30.0, 33.9]	64.8 / 2.5
held-out PhishStorm	stack	0.9562	71.2 [68.7, 72.5]	47.9 [46.0, 49.3]	78.3 / 2.4
held-out PhishStorm	hybrid2	0.9587	73.6 [71.3, 75.0]	56.0 [53.7, 58.3]	82.5 / 3.8
held-out PhishStorm	blend	0.9584	73.5 [71.5, 75.0]	56.5 [53.0, 57.2]	84.0 / 4.8
time slice	LightGBM	0.9496	65.9 [65.6, 66.1]	52.4 [51.5, 53.9]	68.0 / 1.5
time slice	char n-gram + LR	0.9595	64.2 [63.5, 65.0]	38.5 [37.2, 39.7]	68.5 / 1.5
time slice	stack	0.9706	72.4 [71.9, 72.7]	54.5 [53.7, 55.4]	75.3 / 1.4
time slice	hybrid2	0.9753	74.4 [74.1, 75.5]	51.0 [47.7, 53.9]	77.7 / 1.5
time slice	blend	0.9886	84.0 [83.6, 84.8]	62.7 [60.4, 64.0]	83.9 / 0.9

Table 8.2. Held-out source (59,485 URLs, 33,056 domains; no HTML, so the blend equals hybrid2 there) and newest time slice (80,653 URLs, 58,054 domains) (same sources as Table 8.1). The blend row on the held-out source uses the same address model with its own thresholds.

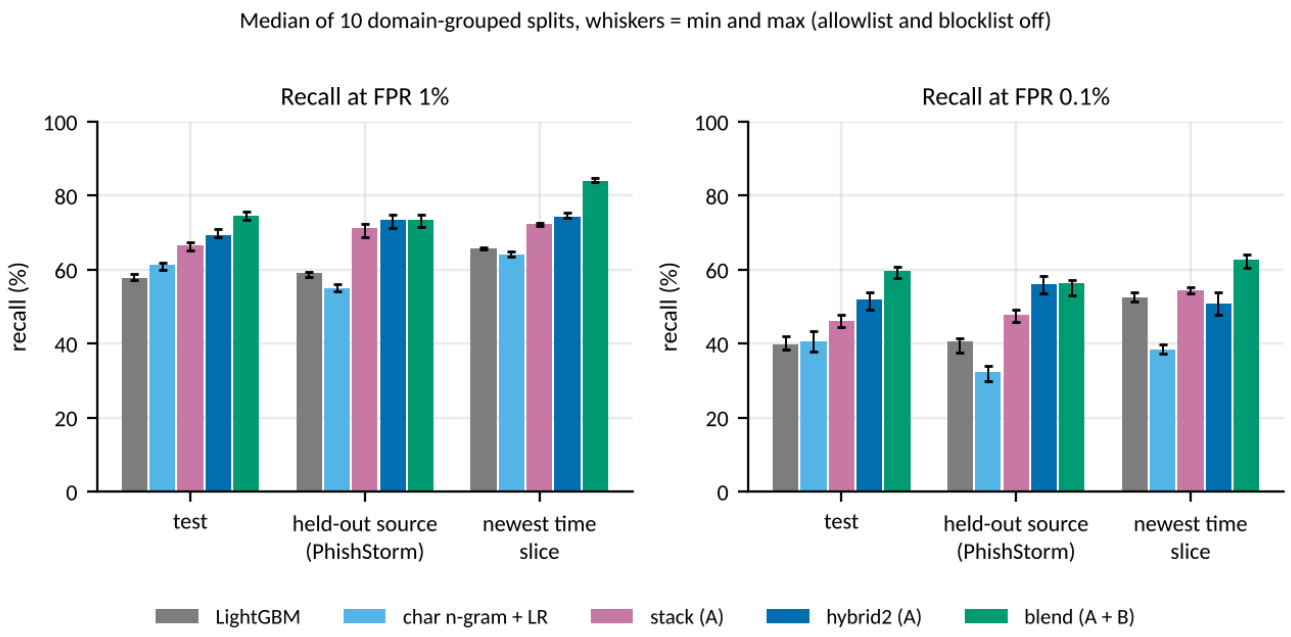


Figure 8.1. Recall at FPR 1% and 0.1% by model and evaluation set (median of 10 splits, whiskers min to max).

Three things stand out. Combining engineered features with text scores lifts tier A by about 12 points at 1% FPR (57.8 to 69.5). The page model adds more on pages with HTML (section 8.4). And the thresholds do not hold on a new source: the cal-side 1% threshold gives 3.8% FPR for hybrid2 and 4.8% for the blend on PhishStorm, and 1.5% on the time slice for hybrid2, against a 1% target. The 0.1% threshold held better on PhishStorm (0.2% and 0.3%, `reports/hybrid2.md`, `reports/blend2.md`).

At the cal thresholds on the test side, precision was 96.2% for hybrid2 and 96.8% for the blend run in `reports/blend.md`, with F1 80.9% and 83.8% (`reports/summary.md`). Precision is high partly because the test side is 26.985% phishing (data card); at real-world prevalence it would be far lower at the same FPR, and for that reason it is not used as a headline. Precision-recall AUC for the base learners ranged from 0.8644 (LightGBM) to 0.8897 (char n-gram) on test (`reports/tier_a.md`).

Drift. The time slice is PhreshPhish only, so the fair drift comparison uses the PhreshPhish rows of the test side. For hybrid2 these give ROC-AUC 0.9820 and recall 77.0 / 60.2 at 1% and 0.1% FPR, against 0.9753 and 74.4 / 51.0 on the newer slice (`reports/hybrid2.md`). The address model lost about three points at 1% and nine at 0.1%.

8.3 What the page adds

rows	model	ROC-AUC	recall @1% FPR	recall @0.1% FPR
test, with HTML (26,011 URLs, 18,334 domains)	address only (hybrid2)	0.9858	83.0 [81.9, 85.3]	66.3 [64.1, 69.1]
test, with HTML	page model (`ab_hybrid2`)	0.9960	94.5 [94.1, 95.7]	86.2 [84.2, 86.9]
time slice, with HTML (62,388 URLs, 46,891 domains)	address only	0.9714	72.0 [71.4, 72.7]	54.4 [51.7, 56.5]
time slice, with HTML	page model	0.9873	85.1 [84.5, 85.7]	66.8 [65.2, 68.3]

Table 8.3. Tier A against tier B on the same rows (`reports/tier_b.md`; thresholds from the calibration rows that also have HTML).

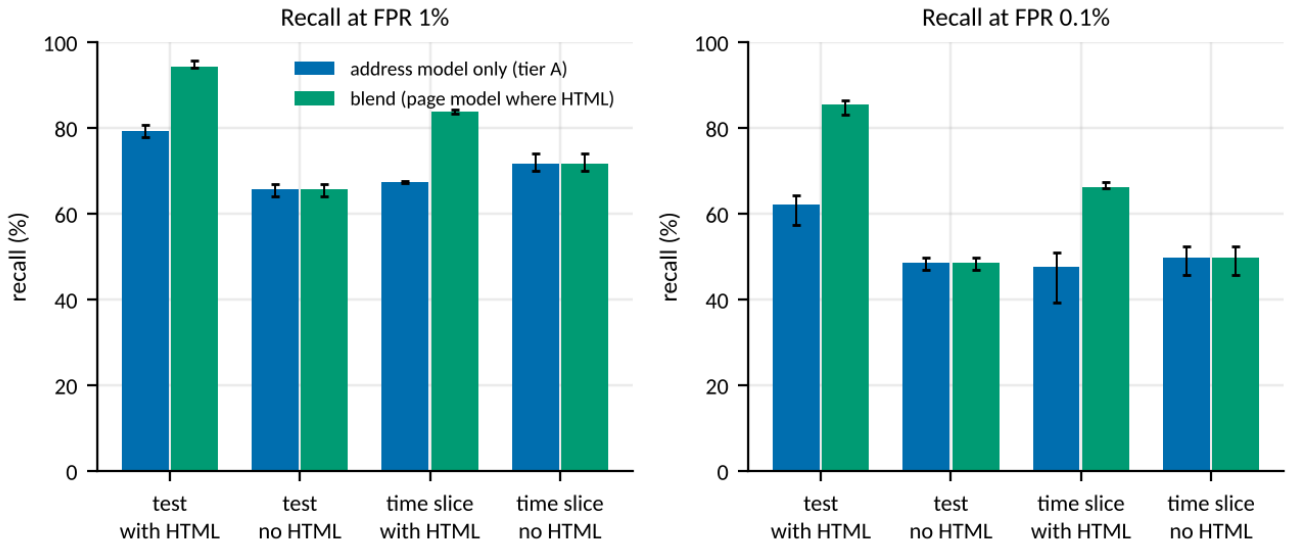


Figure 8.2. Address model against the blend, on URLs with and without usable HTML (`reports/blend2.md`).

Where a page exists, recall at FPR 0.1% rises from 66.3% to 86.2% on test and from 54.4% to 66.8% on the time slice. Where it does not, the blend is the address model and nothing changes (Figure 8.2). The page model lost more recall over time than the address model: for `ab_hybrid2` the PhreshPhish test rows give 94.4 / 86.1 against 85.1 / 66.8 on the time slice (`reports/tier_b.md`).

8.4 Checks on the page model

Test ROC-AUC on HTML rows is above 0.99 (0.9958 for the blend, `reports/blend2.md`), close to the project's leakage tripwire, so it was investigated (`reports/tier_b_checks.md`, 10 splits). A LightGBM trained on shuffled labels scored a mean test ROC-AUC of 0.5071 (not flagged). The 84 page features alone, with no address, reach 0.9853 on test and 0.9573 on the time slice. Removing the 2.15% of test rows (median 558 of 26,011) whose page-feature vector equals a training row's leaves ROC-AUC at 0.9921 (0.9928 with them). Leaving out Phishpedia (phishing only, 2019 to 2020) from training lowers recall at 1% FPR only from 91.7 to 90.4 for `ab_lgbm`, so the model is not simply learning that old pages are phishing. None of this shows leakage. It cannot rule out a PhreshPhish-specific page artefact, because almost all HTML rows are PhreshPhish (24,232 of 26,011 test URLs) and the held-out source has no HTML. The page-model result is, in effect, a single-source result.

Usable HTML is present for 59.9% of test phishing rows but 90.8% of legitimate rows (`reports/blend2_fair.md`), so the HTML subset is not a random sample. Re-weighting each HTML row by the inverse of its estimated probability of having usable HTML (clipped at 20; effective sample sizes 4,788 phishing and 17,855 legitimate rows) lowers the blend's warned recall on HTML rows from 93.0 to 91.9 and its dangerous recall from 85.7 to 82.5. On the time slice the corrected and uncorrected figures differ by less than one point (80.4 to 80.7 warned). The correction assumes HTML loss depends only on the URL features and source; if phishing pages whose HTML survives are easier than those whose HTML does not, even the corrected figure is optimistic.

8.5 The shipped verdict rules

The extension's behaviour is the blend plus rules, so the rules were evaluated as well. `shipped` lets the address model's "dangerous" act before the page loads and then applies the blend; `agree` adds the rule that a suspicious banner on a page with HTML needs both models to agree.

set	rule	phishing warned	legit warned	phishing dangerous	legit dangerous
test	address only	69.7 [68.6, 70.7]	1.0 [0.9, 1.1]	51.5 [50.0, 52.7]	0.1 [0.1, 0.1]
test	agree	73.5 [72.0, 74.3]	0.9 [0.9, 1.1]	60.0 [57.5, 61.0]	0.1 [0.1, 0.1]
test, with HTML	shipped	93.1 [92.1, 94.0]	0.6 [0.5, 0.9]	86.6 [83.9, 87.8]	0.2 [0.1, 0.2]
test, with HTML	agree	88.5 [87.0, 89.5]	0.4 [0.2, 0.4]	86.6 [83.9, 87.8]	0.2 [0.1, 0.2]
test, no HTML	agree	67.1 [64.9, 67.8]	1.1 [1.1, 1.4]	48.8 [45.0, 50.6]	0.1 [0.1, 0.1]
time slice	agree	80.7 [79.8, 81.4]	0.8 [0.7, 0.8]	66.7 [64.6, 68.7]	0.2 [0.2, 0.3]
held-out PhishStorm	agree	84.0 [83.6, 85.2]	4.8 [4.3, 5.8]	65.0 [60.1, 67.7]	0.3 [0.2, 0.5]
natural pages, URL only	agree	n/a	5.3 [4.9, 6.2]	n/a	1.1 [0.7, 1.5]
natural pages, with HTML	shipped	n/a	4.0 [3.2, 4.9]	n/a	1.3 [1.1, 1.6]
natural pages, with HTML	agree	n/a	2.4 [2.1, 3.0]	n/a	1.3 [1.1, 1.6]

Table 8.4. Verdict rules, in % (`reports/blend2_rules.md`; allowlist and blocklist off). "Warned" is suspicious or dangerous.

The `agree` rule cuts the false-alarm rate on natural pages with HTML by about 40% (4.0 to 2.4) at a cost of 4.6 points of recall on test pages with HTML (93.1 to 88.5); it is the shipped rule. It also shows the threshold-transfer problem again: on PhishStorm 4.8% of legitimate URLs are warned. An alternative that uses the highest per-source threshold (`_ws`) lowers that to 3.9% at a cost of 1.3 points of test recall (73.5 to 72.2); it was documented and not shipped, because that threshold is set by one source's calibration slice (`reports/blend2_rules.md`).

8.6 Leave-one-source-out and threshold transfer

Leave-one-source-out trains and calibrates without one source and scores all of it. It was run for tier A with the original base learners on splits 0 to 2 only (`reports/tables/loso_summary.csv`), not on 10 splits and not for hybrid2, tier B or the blend. The baseline is `tld_risk` alone.

left-out source	URLs / domains	model	ROC-AUC	recall @1% FPR	cal 1%: recall / FPR	cal 0.1%: recall / FPR
PhreshPhish	147,232 / 106,917	LightGBM	0.8603	29.5	73.8 / 20.1	53.5 / 5.6
PhreshPhish		baseline	0.7945	43.4	44.7 / 1.3	38.4 / 0.3
PhiUSIIL	171,826 / 157,550	LightGBM	0.8013	35.3	38.7 / 1.5	21.8 / 0.1
URL-Phish	91,041 / 84,130	LightGBM	0.9350	51.3	71.1 / 4.4	52.4 / 1.1
Live feeds	245 / 97	LightGBM	0.9056	36.1	61.7 / 9.8	29.3 / 0.0

Table 8.5. Leave-one-source-out, tier A, 3 splits (`reports/loso.md`). Phishpedia and Phish-Blitz have only phishing rows and give no FPR.

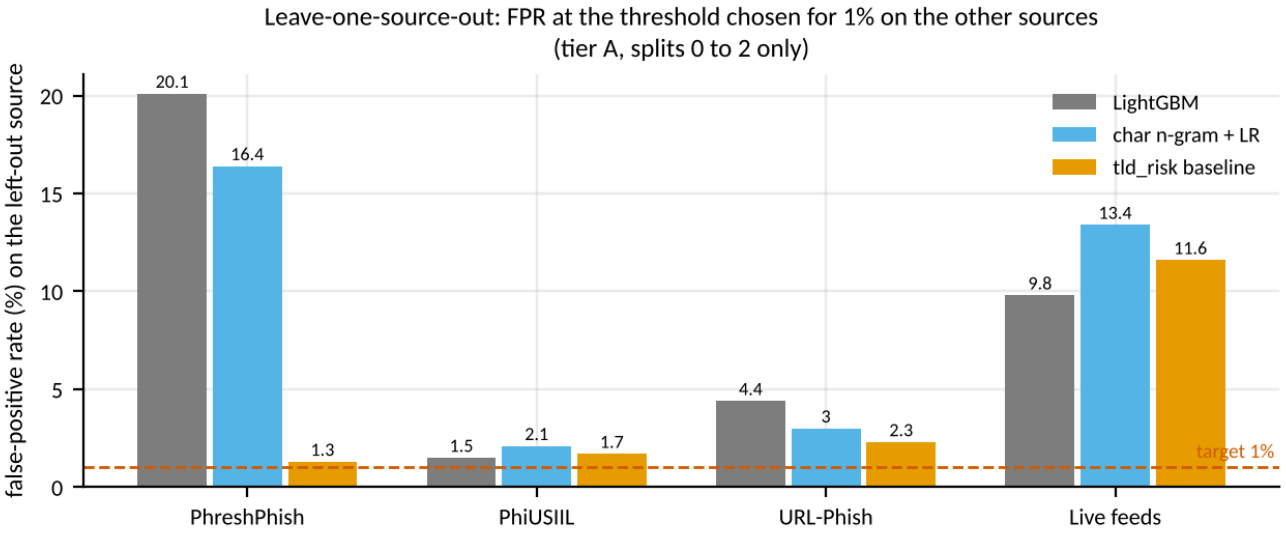


Figure 8.3. False-positive rate on the left-out source at the threshold chosen for 1% on the other sources.

This is the weakest result of the project. Holding out PhreshPhish, the 1% threshold produced 20.1% FPR, and the single-feature `tld_risk` baseline (43.4% recall at 1% FPR) beat both trained models (29.5% and 8.2%). The models partly learn source style, and thresholds tuned on some sources do not transfer to another. The live-feed row rests on 245 URLs and is shown for completeness only.

8.7 Depth and representation checks

The representation check rewrites test URLs to `https://www.<registrable domain>`, which removes the path and subdomain cues an attacker can change cheaply.

model	ROC-AUC raw	ROC-AUC normalised	recall @1% normalised	recall @0.1% normalised
LightGBM	0.9182	0.8291 [0.8265, 0.8312]	36.7 [36.2, 37.8]	25.7 [24.6, 26.1]
char n-gram + LR	0.9363	0.8559 [0.8542, 0.8587]	40.3 [39.5, 41.4]	24.2 [22.1, 24.6]
hybrid2	0.9536	0.8697 [0.8682, 0.8715]	44.3 [43.7, 45.0]	29.8 [28.1, 30.7]
blend (page features kept)	0.9618	0.9132 [0.9108, 0.9167]	53.8 [52.9, 54.8]	39.0 [37.4, 42.3]

Table 8.6. Representation check on 71,063 normalised test URLs from 71,044 domains (`reports/tier_a.md`, `reports/hybrid2_checks.md`, `reports/blend_checks.md`). The blend rows come from an earlier blend run (raw ROC-AUC 0.9618, recall 73.7 / 58.6) than Table 8.1.

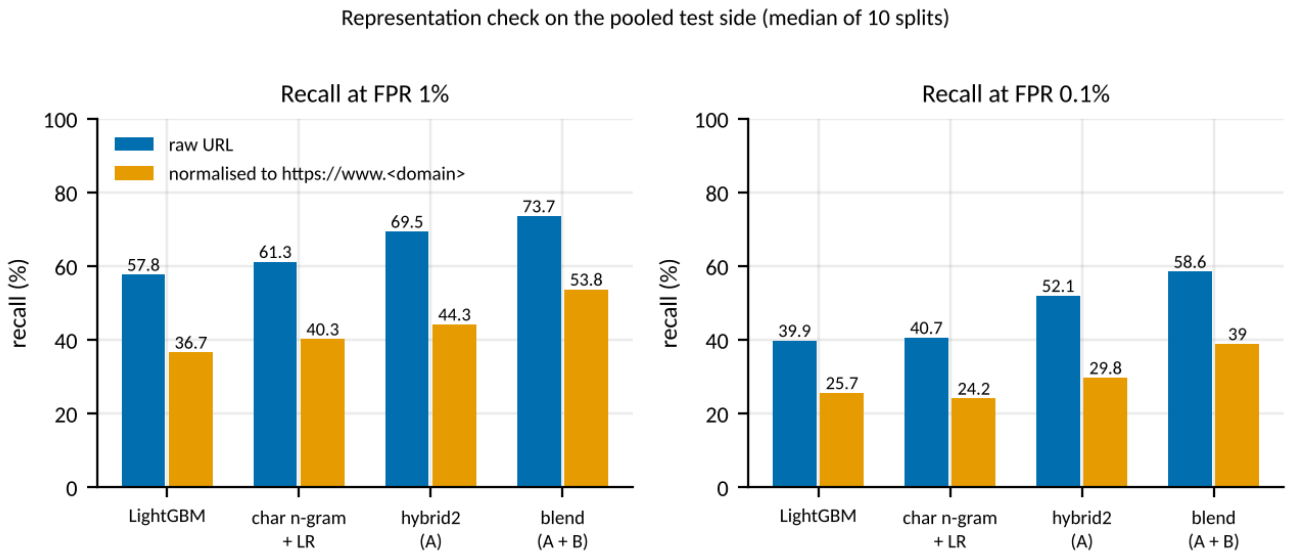


Figure 8.4. Raw against normalised URLs: recall at FPR 1% and 0.1%.

The gap is large. For the blend, on rows with HTML the ROC-AUC falls only from 0.9958 to 0.9830, so where a page exists it carries most of the signal; on rows without HTML it falls from 0.9335 to 0.8687. One split gives 0% recall at 0.1% FPR on the normalised HTML rows (median 65.7%) (`reports/blend_checks.md`).

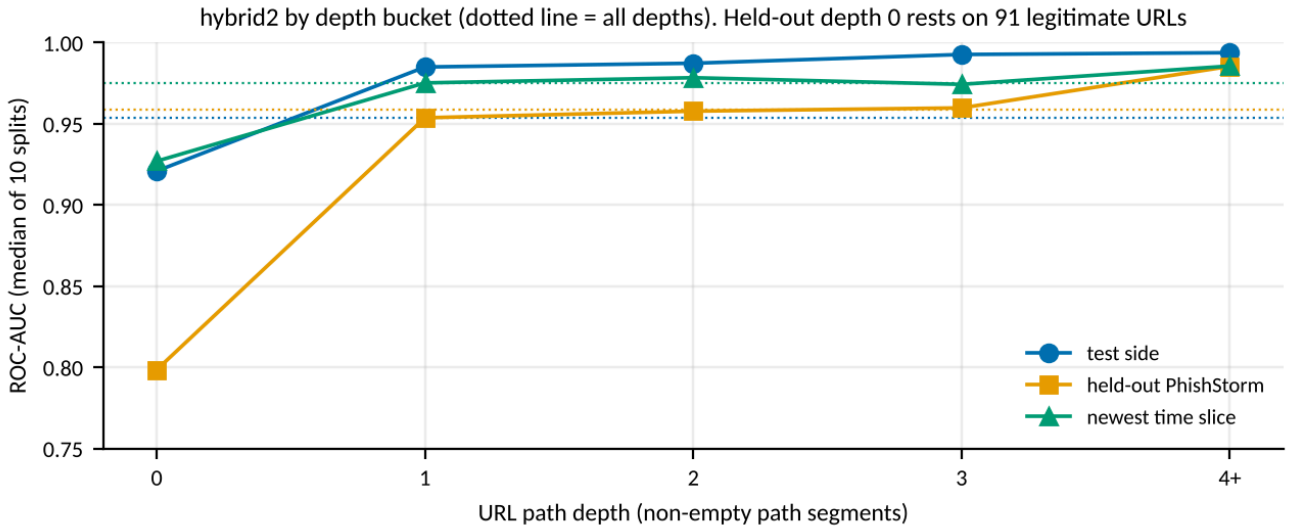


Figure 8.5. Shipped address model (hybrid2), ROC-AUC by URL depth bucket on three sets.

The depth-matched view (Figure 8.5, `reports/hybrid2_depth.md`) shows that the result does not rest on depth alone. Depth 0 (bare domains) is the weakest bucket on every set: 0.9209 on test, 0.9270 on the time slice and 0.7985 on PhishStorm, where it holds only 91 legitimate URLs and the cal 1% threshold flags 30.2% of them. On PhishStorm depths 1 to 3 give 0.954 to 0.960, about the overall 0.9587, so the held-out result is not an artefact of PhishStorm's depth imbalance.

8.8 False alarms on naturally browsed legitimate pages

Every model is scored on the 2,555 natural pages at its cal-side thresholds.

model	threshold from	1% target	0.1% target
LightGBM	whole cal side	4.5 [3.9, 5.1]	0.6 [0.3, 0.9]
LightGBM	PhreshPhish cal only	3.4 [2.9, 4.2]	0.4 [0.2, 0.6]
XGBoost	whole cal side	4.5 [4.0, 5.3]	0.6 [0.4, 0.8]
char n-gram + LR	whole cal side	5.1 [4.5, 5.4]	0.9 [0.5, 1.1]
first hybrid	whole cal side	5.1 [4.6, 5.4]	0.9 [0.5, 1.1]
hybrid2	whole cal side	4.3 [3.9, 4.9]	0.8 [0.5, 1.1]
blend, URLs only	pooled blend cal	5.3 [4.1, 5.9]	1.0 [0.8, 1.6]
blend, pages with HTML (2,226)	pooled blend cal	3.6 [2.8, 5.1]	0.7 [0.2, 1.8]

Table 8.7. Share of natural legitimate pages flagged, % (`reports/ood_fpr.md`, `reports/blend_ood.md`; 562 sites, 512 with HTML).

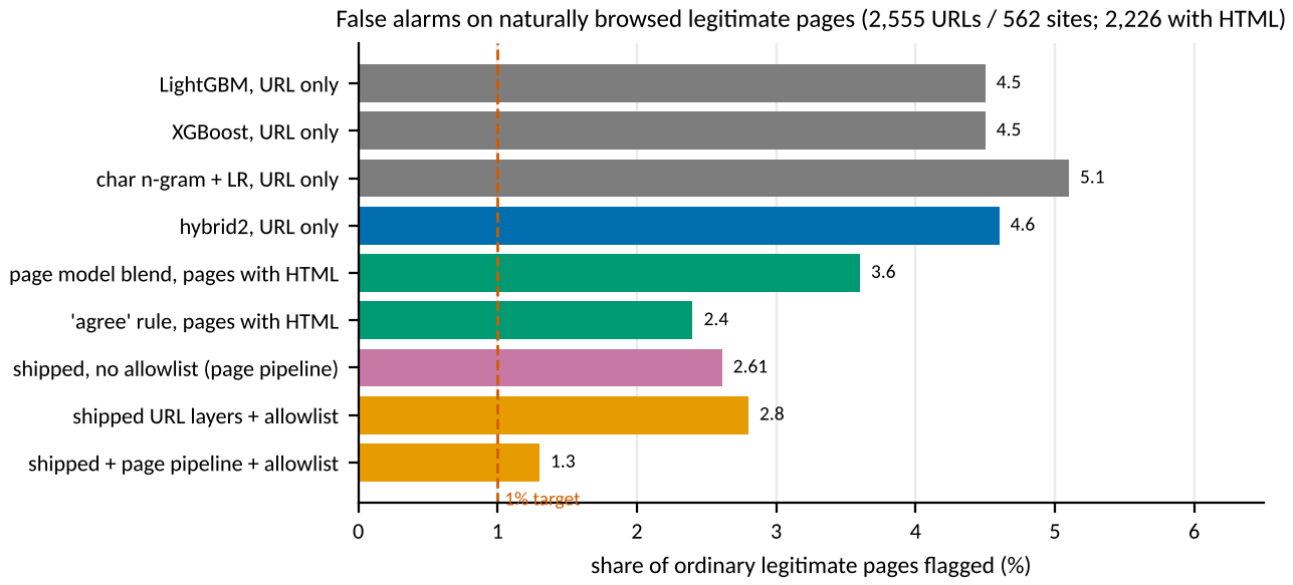


Figure 8.6. Natural-page false alarms by model and by layer set (`reports/ood_fpr.md`, `reports/blend2_rules.md`, `reports/shipped_verdicts.md`, `reports/page_fixes_fpr.md`).

No model flags every natural page, so the homepage shortcut has not returned. Even so, at the 1% target between 3.4% and 5.3% of natural pages are flagged, well above the target. Taking thresholds from the PhreshPhish part of the calibration side (real browsing telemetry) lowers the rate for every model, which suggests that part of the excess comes from the threshold rather than the model.

The shipped pipeline does better because of the allowlist and the page model. Offline, with the allowlist and the address model at both thresholds, 71 of the 2,555 natural pages are flagged (2.8%; 64 suspicious, 7 dangerous; 38 domains), and 71 of the 1,483 pages on sites outside the top 50,000 (4.8%) (`reports/shipped_verdicts.md`). With the page model, the password-field rules and the allowlist, 29 of 2,226 readable pages with HTML are warned (1.30%) and 11 are dangerous (0.49%); with the allowlist off the figures are 2.61% and 1.17% (`reports/page_fixes_fpr.md`). The rule that stops an address-only “dangerous” from opening the warning page cut the number of pages that would have been blocked from 11 to 6 of 2,373 with the allowlist on (30 to 23 with it off), without changing the warned share (`reports/address_only_blocks.md`). All of these are measured on stored HTML, not on a live page, and the natural set is Tranco-seeded.

8.8.1 After retraining with deep legitimate pages of popular sites

The address model's false alarms were attributed to benign training URLs that were mostly bare homepages (an explanation from the project log, not a separate test). A crawl of inner pages of popular sites (14,026 pages, robots.txt respected; the 2,555 natural pages were excluded from it) was added to the pool, the brand list was extended from 772 to 899 names, and the shipped split-0 model was retrained and exported on the project PC on 2026-10-21 (`docs/PROGRESS.md`, entry "Retrain on the pool with the popular-sites crawl").

measure	before retrain	after retrain
shipped extension, natural pages flagged (of 2,555)	72 (2.82%)	8 (0.3%, none dangerous)
address model alone, natural pages flagged	115 (4.5%)	19 (0.7%)
sites outside the top 50,000, flagged	71 of 1,483 (4.8%)	8 of 1,579 (0.5%)
blend recall at FPR 1%, test	73.7 (10-split median)	74.2 (split 0)
blend recall at FPR 1%, held-out PhishStorm	71.5 (10-split median)	71.0 (split 0)
blend recall at FPR 1%, newest time slice	84.2 (10-split median)	81.7 (split 0)

Table 8.7b. Natural-page false alarms and recall before and after the retrain (`docs/PROGRESS.md`, 2026-10-21 entry; PC reports `reports/shipped_verdicts.md` and `data/models/reports_split0/blend2.md`, which are not in the repository copy).

Read these numbers with care. The "after" recall figures come from one split, the "before" figures are medians over ten, so the two are not strictly comparable. The crawl hosts and the natural pages both come from popular-site lists, so some of the drop may reflect that similarity. The brand-list change went in together with the new data, so the two effects are not separated. The false-alarm rate on PhishStorm at the cal-side 1% threshold stays at 3.2%. The ten-split tables elsewhere in this chapter were produced before the retrain and have not been refreshed.

8.9 Negative controls

model	splits	mean ROC-AUC	min	max	flagged
LightGBM (A)	10	0.4941	0.4757	0.5138	no
char n-gram + LR (A)	10	0.4997	0.4963	0.5093	no
hybrid2 (A)	10	0.4940	0.4749	0.5125	no
tier B (`ab_lgbm`, HTML rows)	10	0.5071	0.4698 (test)	0.5261 (test)	no
blend, with calibration	10	0.5358			no
tier C LightGBM	10	0.5191	0.4273	0.5976	no

Table 8.8. Shuffled-label controls, flag rule |mean AUC - 0.5| > 0.05 (`reports/tier_a.md`, `reports/hybrid2_checks.md`, `reports/tier_b_checks.md`, `reports/blend_checks.md`, `reports/tier_c.md`). The blend figure is the 10-split mean of the test AUC; the report prints a median of 0.5328.

No condition is flagged. The blend control stays slightly above 0.5. The likely reason is that the isotonic step is fitted with the true calibration labels and can map source differences in a shuffled score onto the sources' phishing rates; this is inferred, not measured (`reports/blend_checks.md`). Controls were trained without sample weights, because with the

heavy per-source weights they drifted below 0.5 through a leaf-size artefact (`docs/PROGRESS.md`, 2026-10-12). Tier C is the control with the widest range because its test side has only 24 phishing hosts.

8.10 Robustness to URL evasion

Six evasion families were applied to test URLs (`attacks/`): look-alike letters (ASCII or punycode), stacked subdomains, a brand name in the path, long path and query padding, wrapping in a public shortener, and a swapped TLD with hyphenation. The shipped model (`base`) was compared with the same model trained on variants of both classes (`adv`: 12% of training rows, one family held out per split). Six splits were run instead of ten, because each split trains two models. Evaluation used 4,000 phishing and 6,000 legitimate test URLs per family; thresholds are the clean cal-side 1% and 0.1% points (`reports/robustness.md`).

On clean URLs, `base` has recall 69.6 / 52.1 at FPR 1% / 0.1% and `adv` has 68.6 / 50.2, so adversarial training costs about one point.

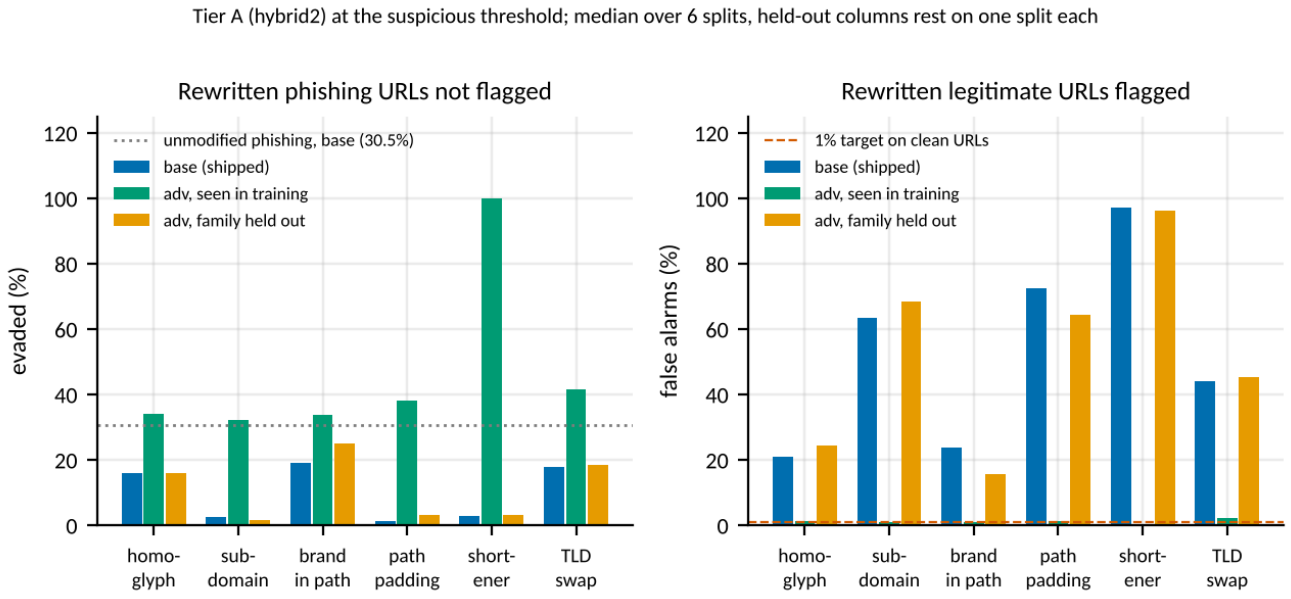


Figure 8.7. Evasion rate and false alarms before and after adversarial training, per family, at the suspicious threshold.

The result is the opposite of the usual evasion story. The shipped model is hard to evade with these rewrites (2.4% evasion for stacked subdomains, 1.3% for padding, 2.9% for shortener wrapping, against 30.5% for unmodified phishing), but only because it treats the rewrite as suspicious: it also flags 63.5%, 72.5% and 97.2% of legitimate URLs rewritten the same way. Long legitimate URLs that were not rewritten are not a problem (0.7% flagged). Training on variants of both classes removes the false alarms for the families it saw (1.1% to 2.1%; 0.0% for shortener) and returns evasion to the clean rate (32% to 41%; shortener 100%, because a short link carries nothing for a URL model). A family the model never saw behaves like the base model on both sides, and each held-out column rests on one split. The apparent robustness of the base model is therefore over-suspicion rather than understanding, and the extension needs its other layers for real traffic: shorteners are expanded and scored at the target, and the allowlist and blocklists carry sites that a URL model would flag. The rewrites are synthetic (random lure words, brands from the project's own list), so real attacker URLs may differ. Robustness of tier B was not measured.

8.11 Tier C

Tier C uses live-feed hosts captured within two hours of first being seen (median 0.1 hours for phishing, 0.2 for legitimate). After removing hosting-platform hosts, hosts answered by the collector PC's filtering resolver, hosts that did

not resolve, late captures and hosts whose domain is already in the tier A pool, 1,369 legitimate hosts (741 domains) and 119 phishing hosts (91 domains) remained (`reports/tier_c.md`).

model	test hosts / domains	pos / neg	ROC-AUC	recall @1% FPR	recall @0.1% FPR
tier C LightGBM	299 / 169	24 / 275	0.7781 [0.6226, 0.9048]	25.0 [16.7, 66.7]*	20.8 [8.3, 58.3]*
domain age only	299 / 169	24 / 275	0.6143 [0.5255, 0.6814]	0.0*	0.0*
tier A model on the bare host	299 / 169	24 / 275	0.7971 [0.7089, 0.8692]	6.2 [0.0, 12.5]*	4.2 [0.0, 8.3]*
tier C + tier A host score	299 / 169	24 / 275	0.8659 [0.6577, 0.9091]	25.0 [16.7, 54.2]*	22.9 [8.3, 37.5]*
tier C, shuffled labels	299 / 169	24 / 275	0.5298 [0.4273, 0.5976]	0.0*	0.0*

Table 8.9. Tier C, median of 10 domain-grouped splits; `*` granularity-limited (`reports/tier_c.md`).

With 24 phishing and 275 legitimate test hosts, every fixed-FPR point is granularity-limited (the 1% budget allows two false positives) and should be read as unmeasured. The usable result is the ranking: tier C separates the classes better than domain age alone (0.7781 against 0.6143) and adds to the address model (0.8659 combined against 0.7971). Top features by gain are `days_since_rdap_change` (16.1%), `domain_age_days` (13.2%) and `cert_age_days` (11.7%). Legitimate hosts were sampled from Tranco and arrive as `<domain>` and `www.<domain>` pairs, so collection method may leak into host-form features. The time check was skipped because the eligible rows span two capture days. Tier C is not wired into the backend: the backend's run-time checks are the hand-set rules in section 7.5.

8.12 Real-world and stress tests

The environment that built the system cannot browse, so these are measurements on feeds, stored pages and local test pages, not a browsing study. Results are before and after fixes, from `docs/PROGRESS.md` unless a report is named.

- **First real-site use (2026-10-13).** The author's first browsing test found `web.whatsapp.com` called dangerous: the model scored it 0.991 against a 0.990 threshold, and without a backend the extension had no allowlist. The address model alone flags 161 of the Tranco top 1,000 homepages. This led to the bundled allowlist (section 7.3).
- **URL layer on fresh feed URLs (2026-10-14).** On 200 active phishing hosts from Phishing.Database and 104 regional brand URLs, 62.5% of phishing was warned before the zone fix, 12 model-flagged hosts were silenced by allowlisted hosting domains, and 0 of 104 brand URLs were flagged. After zones were learned from phishing history, 68.5% were warned (73 dangerous, 64 suspicious), none were silenced, and 0 of 104 brand URLs were flagged. The 200 hosts were left out of the zone evidence.
- **Stress test (2026-10-15).** A headless-browser and Node test with fresh feed phishing, hosting-platform phishing, six evasion families, long URLs and 138 legitimate login pages found five real gaps (a very long URL threw an error that disabled every check; allowlisted sites were never page-checked; short links failed both ways; redirect targets were judged only after render; the page could choose which address was scored). All five were fixed. Before to after: hosting-platform phishing warned 81.0% to 85.9%, OpenPhish 92.3% to 92.7%, Phishing.Database active 82.0% to 82.2%; natural legitimate pages flagged unchanged at 2.7%; legitimate links wrapped in a shortener flagged 29.6% to 0.0% and phishing wrapped in a shortener 24.6% to 0.0% at the address, now decided at the target (the last step is inferred from the code path, not measured in a browser). Part of the Phishing.Database sample overlaps the data the zones were learned from, so that row flatters the system.

- **Tester rounds on local attack pages (2026-10-19).** A separate test thread ran 35 attack cases in real Chromium on locally served pages (late-injected forms, frames, shadow roots, `replaceState`, back-to-safety). The first build passed 21 of 35, the next 31 of 35, then 34 of 35, and after the fourth round of fixes 35 of 35 in four full runs in a row. After the 2026-10-20 address-only change the same harness gave 33 of 35, with the two trust cases failing only because the harness's stub serves lists without the signed manifest. These are constructed pages, so they test the logic, not detection of live phishing.
- **Address rules and tenant sites.** Section 7.3 and 7.4 report their own before and after counts.

Blocklist coverage on the corpus is not measurable, because the test URLs come from datasets collected in other periods than the live feeds. It can only be measured forward, as the share of newly listed feed URLs whose domain was already on an earlier list, once the collector has run for several days. This is not measured.

9. Limitations and what is not done

What the results do and do not support.

- **Thresholds do not transfer across sources.** The cal-side 1% threshold gives 20.1% FPR when PhreshPhish is held out (Table 8.5), 3.8% to 4.8% on PhishStorm (Table 8.2), and 3.4% to 5.3% on natural pages (Table 8.7), against a 1% target. A conservative suspicious threshold or per-source calibration is the obvious next step; one such variant was tested and not shipped (section 8.5).
- **The bare-domain ceiling.** PhiUSIIL phishing is mostly bare domains, and hybrid2 reached only 38.5% recall at FPR 0.1% on PhiUSIIL test rows (`reports/hybrid2.md`). Without a path or a page the domain name carries little signal. Normalising URLs lowers recall at 1% FPR from 69.5 to 44.3 (Table 8.6).
- **The page model is effectively single-source.** Almost all HTML rows come from PhreshPhish, the held-out source has no HTML, and a PhreshPhish-specific page artefact cannot be excluded. Survivorship in saved HTML (taken-down or cloaked pages are missing) is handled only by an assumption-based correction (section 8.4). Dated phishing HTML (Phishpedia, 2019 to 2020) differs from current pages, and the page model drifted more than the address model.
- **Evaluation coverage is uneven.** Leave-one-source-out used three splits and the original tier A models; robustness used six splits and synthetic rewrites; tier B robustness and a tier B held-out source do not exist; the extra hybrid2 text scorers were chosen on calibration slices that are also used for thresholds.
- **Tier C is a pilot.** 119 phishing hosts from 91 domains, every fixed-FPR point granularity-limited, no second live source, Tranco-sampled legitimate hosts, and no time check. The backend does not run a tier C model.
- **The false-alarm numbers are optimistic.** The natural pages are Tranco-seeded, most were measured on stored HTML, and the brand rules and several fixes were tuned after the natural set was seen.
- **The "blend" has two runs.** The blend and hybrid2 appear with slightly different numbers in `reports/blend.md` and `reports/blend2.md` (for example blend recall at 1% FPR on test 73.7 against 74.6), because they are separate training runs; Table 8.1 uses the newer one and Table 8.6 the older one.

What is not done or not measured.

- **Browsing study.** No systematic test on real sites with screenshots of false alarms. The sandbox cannot open real sites; the author's own use found one false alarm (section 8.12).
- **Live backend lookups.** Backend response times and a live RDAP, DNS and TLS lookup against real feeds have not been measured; the extension's backend answers are placeholder rules.
- **Safe Browsing and the language-model layer.** Google Safe Browsing is not integrated, and the optional second-opinion layer was not built. List signatures are off until a key is made on the backend machine.
- **Model refresh.** The shipped model was retrained on 2026-10-21 for split 0 with the 899-name brand list and the popular-sites crawl (section 8.8.1). The ten-split tables in this chapter still describe the earlier model, and the effect of the new data and of the brand list is not separated.
- **Blocklist coverage** (section 8.12), **screenshots** and **demo video** are not done.

Out of scope. Phishing hosted on trusted platforms is masked in the URL tier and removed from tier C, so it is invisible to those tiers. Phishing on compromised legitimate sites and behind cloaking is largely invisible to all three. A learn-from-mistakes loop was designed (opt-in reports confirmed by an independent signal before they become labels, then a scheduled retrain that ships only if the held-out source, time slice and natural-page check are no worse) but not built; users can currently report only a false alarm, with the domain alone.

Known weaknesses of the extension. A dangerous page can start loading before the warning replaces it, because `webNavigation` cannot block a navigation. A hosting zone that nobody has abused in the data is still trusted until it appears in a feed. A phishing page whose address alone is dangerous now renders with the banner until the page check completes (at most 5 seconds) before the warning page replaces it (`docs/PROGRESS.md`, 2026-10-20).

10. Conclusion and future work

10.1 What was achieved

- **An evaluation study.** A URL model's score depends heavily on URL form: it holds across 25 source cells, four model families and ten splits. Transfer to independent datasets stays above chance (holds). Source dependence holds in part and the natural-page false-alarm gap holds in part. The share of the fall that sits in the host name and the size of the split-to-split spread did not hold across the cells, and an extra check ties the first to how the raw benchmark is prepared.
- **A working detector.** PhishDrift runs as a Chrome extension with a FastAPI backend and layered checks (allowlist, blocklist, URL model, page model, domain checks), evaluated with recall at fixed false-alarm rates, a held-out source, a newest time slice and natural-page false alarms (Chapter 8).
- **A reproducible record.** Every reported number comes from a script in the repository that writes to `reports/`; seeds are fixed; the study's plan and consistency rule were frozen before the run.

10.2 Limitations

See Sections 4.12 and Chapter 9. In short: public benchmarks only, two of them old or homepage-heavy; the HTML part rests on two sources with uneven coverage; one shuffled-label control is flagged; platform masking showed no effect in a test that rests on one legitimate source; tier C of the detector rests on a small live capture; and the extension has been checked in test harnesses and on stored pages, not in a long field trial.

10.3 Future work

1. Add PhishTank and further live-feed sources to the study, and re-run it with hosting-platform domains kept so platform masking can be tested.
2. Test the benchmark-preparation explanation for the host-share result on more datasets.
3. Add a held-out source that has page HTML, so the page model's gain can be measured out of source.
4. Grow the tier C capture over more days, and measure blocklist coverage forward in time.
5. Optional second-opinion layer for the grey zone (planned, not built).

References

1. A. Hannousse and S. Yahiouche, "Towards benchmark datasets for machine learning based website phishing detection", *Engineering Applications of Artificial Intelligence*, 2021.
2. A. Prasad and S. Chandra, "PhiUSIIL: a diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning", *Computers & Security*, 2024.
3. PhreshPhish dataset, Hugging Face `phreshphish/phreshphish`.
4. URL-Phish dataset (Mendeley Data); PhishStorm dataset (Aalto University); Phish-Blitz and Phishpedia datasets as released by their authors.
5. G. Ke et al., "LightGBM: a highly efficient gradient boosting decision tree", *NeurIPS*, 2017.
6. T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system", *KDD*, 2016.
7. V. Le Pochat et al., "Tranco: a research-oriented top sites ranking hardened against manipulation", *NDSS*, 2019.
8. OpenPhish community feed; Phishing.Database (MIT licence); Google Safe Browsing Update API.

Citation details should be checked against the originals before submission.